



汇编语言与微机接口技术

第二章 微处理器

2021年8月23日



信息与软件工程学院

School of Information and Software Engineering

主要内容

1 特点

2 主要引线功能和内部结构

3 内部寄存器

4 实地址模式下的存储器分段

5 总线时序

2.1

● 特点及工作模式

2.1 特点及工作模式

» 8088/8086 CPU的特点

CPU内部结构

A

采用并行流水线工作方式

将CPU划分成两个功能部分并设置指令预取队列，
实现流水线工作

存储器寻址部分

B

对内存空间实行分段管理

将内存分为4个段并设置段地址寄存器，以实现
对1MB空间的寻址

CS、DS、ES、SS

C

支持多处理器系统

工作模式

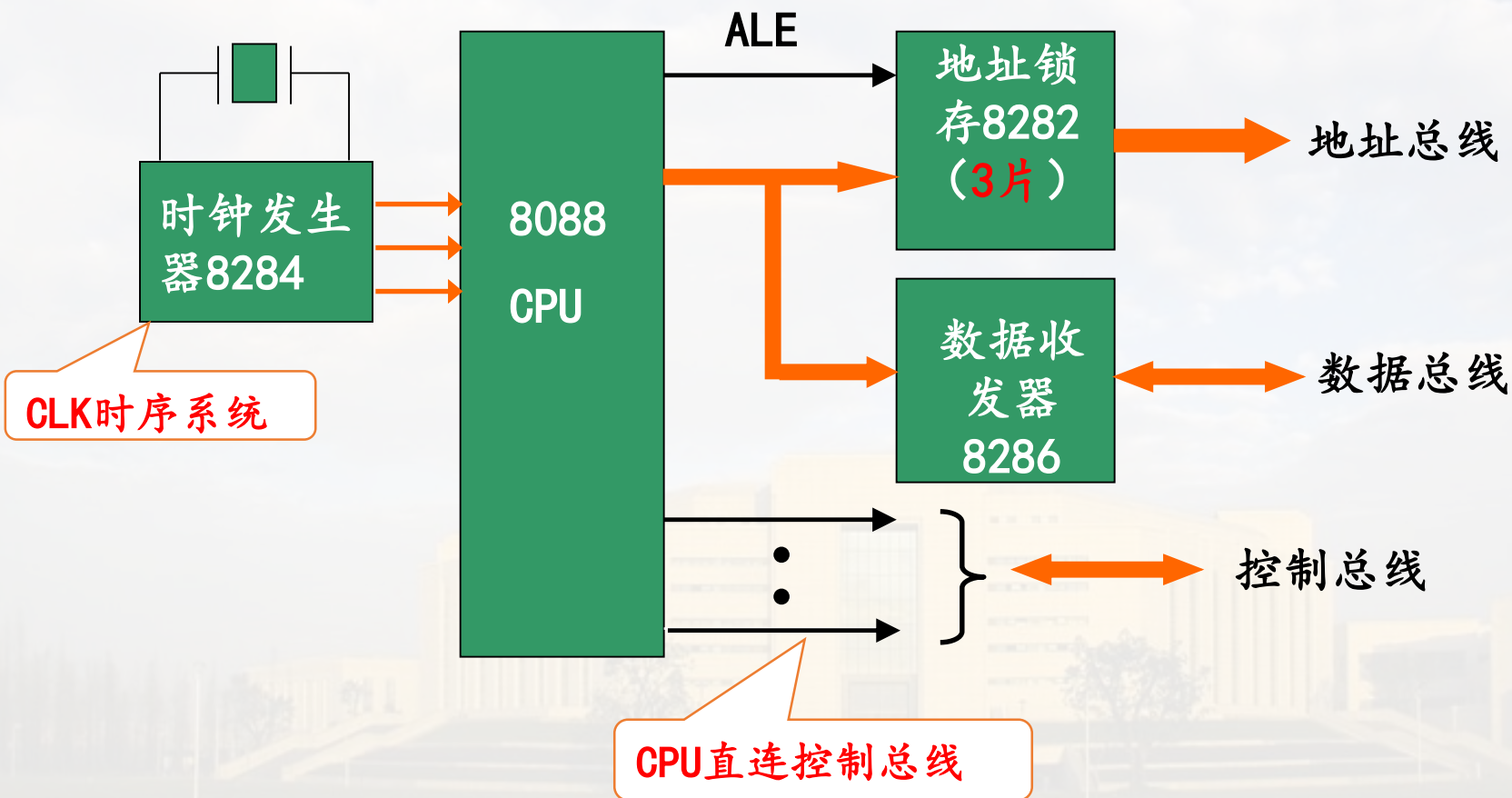
2.1 特点及工作模式

8088CPU的两种工作模式



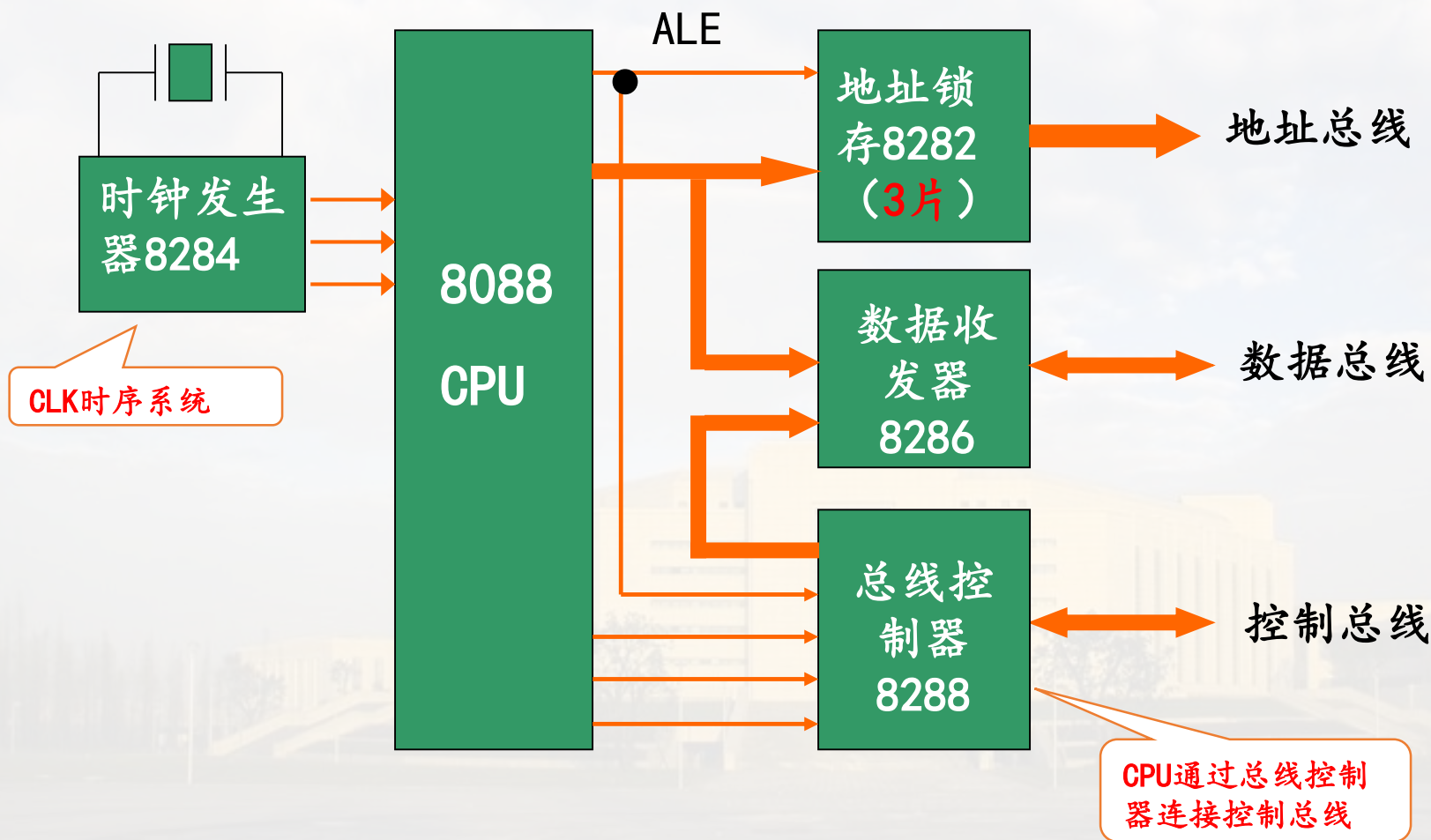
2.1 特点及工作模式

» 最小模式下的总线连接示意图



2.1 特点及工作模式

最大模式下的总线连接示意图



2.1 特点及工作模式

» 两种工作模式的选择

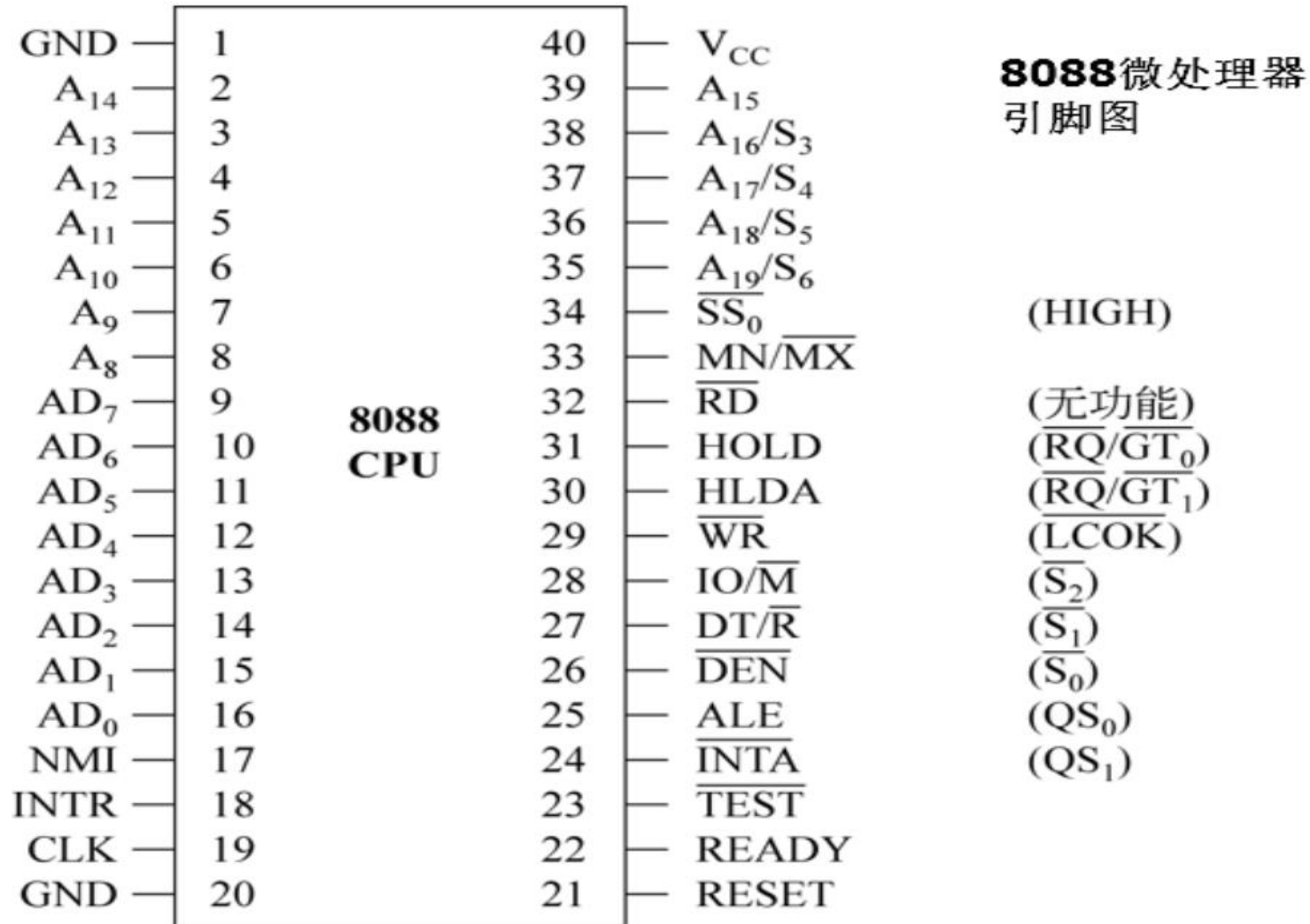


8088是工作在最小还是最大模式
由 $MN/\overline{MX}$ 线的状态决定。

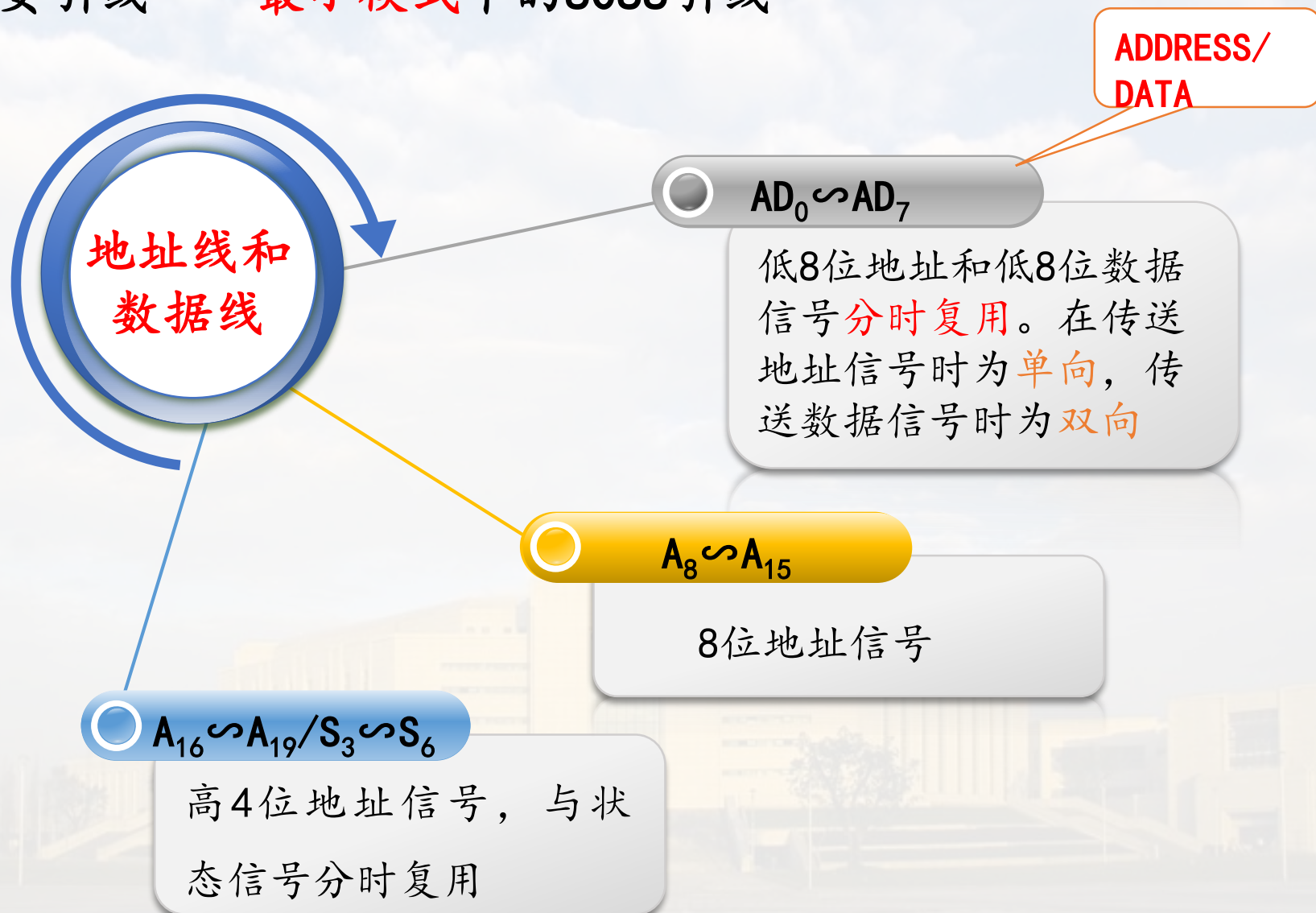
2.2

8088/8086的 引线及功能

2.2 8088/8086的引线及功能



» 主要引线——最小模式下的8088引线



2.2 8088/8086的引线及功能

主要引线——主要的控制和状态信号

写信号

$\overline{WR}$

Data Transmission/
Reception

DT/ $\overline{R}$

数据收发器的传送方向控制，为“1”时CPU向存储器或I/O传送，否则为反向；

读信号

$\overline{RD}$

ALE

地址锁存信号，当其为高时表示地址线上地址有效。一般将地址锁存到一个锁存器中；

为“0”表示访问内存，
为“1”表示访问接口；

IO/ $\overline{M}$

RESET

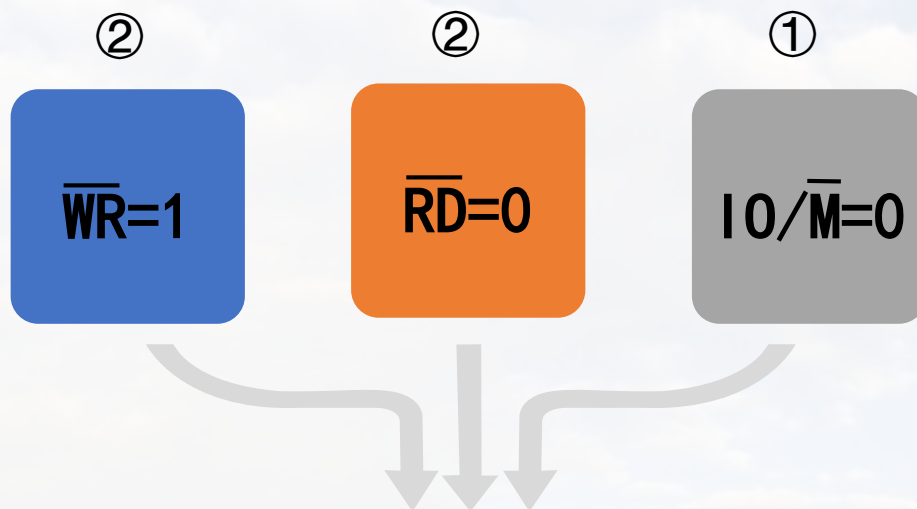
复位信号。当其为高时将完成CPU内部复位。复位后CPU内部寄存器见教材表2-2.

低电平有效时，数据总线上数据有效，允许进行读/写操作；

$\overline{DEN}$

Data Enable

» 主要引线——主要的控制和状态信号——举例



CPU当前正在进行读存储器操作

2.2 8088/8086的引线及功能

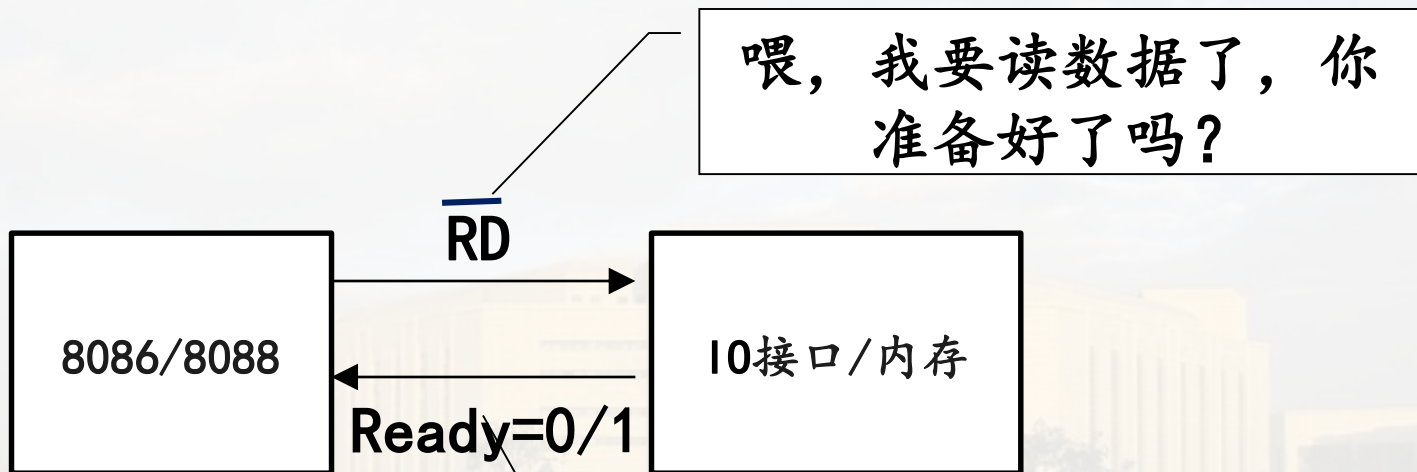
» 主要引线—— READY



外部同步控制输入信号，高电平有效



8088与内存/外设之间在一个总线周期内的时钟配合信号



喂，我要读数据了，你准备好了吗？

0：等等我，我还有点问题
1：可以工作

» 主要引线——中断请求和响应信号



可屏蔽中断请
求输入端



非屏蔽中断请
求输入端



中断响应输出端

» 主要引线——总线保持信号



HOLD

如：在DMA方式下

总线保持请求信号输入端。当CPU以外的其它设备要求占用总线时，通过该引脚向CPU发出请求。

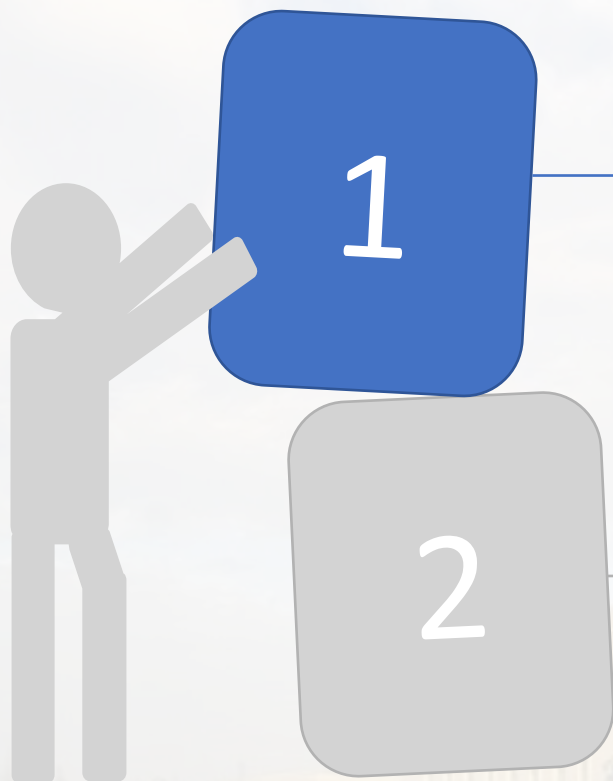


HLDA

如：在DMA方式下

总线保持响应信号输出端。CPU对HOLD信号的响应信号。

» 8088和8086CPU引线的差异



数据总线宽度不同

8088的外部总线宽度是8位；

8086的外部总线宽度是16位。

访问存储器和I/O控制的信号
含义不同

8088—— $\overline{IO}/\overline{M}=0$ 表示访问内存；

8086—— $\overline{IO}/M=1$ 表示访问内存。

2.3

8088/8086的 内部结构

2.3 8088/8086的内部结构



组成

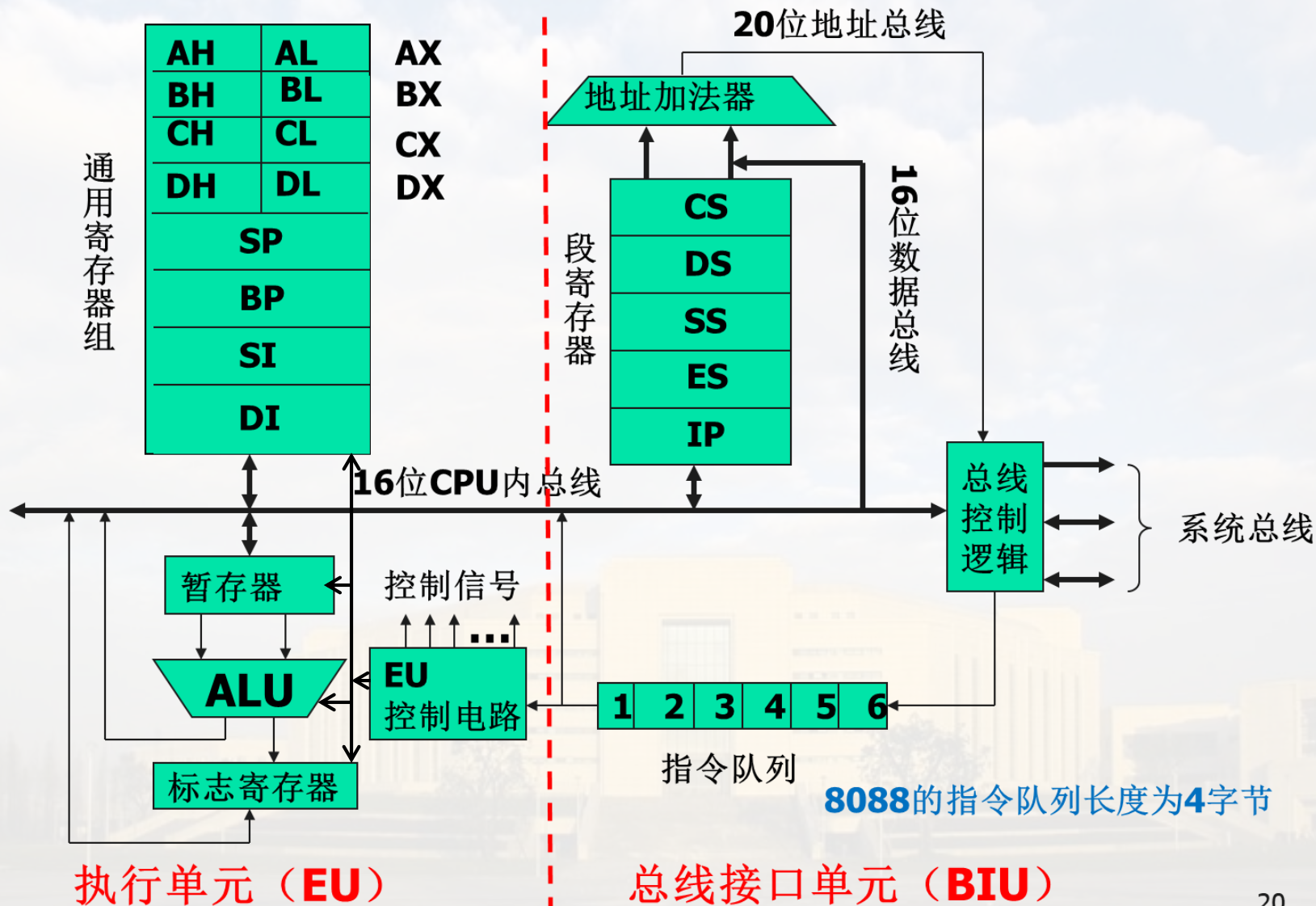


EU: Execution Unit

BIU: Bus Interface Unit

2.3 8088/8086的内部结构

8086CPU结构



» 执行单元



» 执行单元

功能 → 指令的执行

▶ 指令译码 → 在译码电路中完成

▶ 指令执行 → 在ALU中完成

▶ 暂存中间运算结果 → 在通用寄存器中

▶ 保存运算结果特征 → 在标志寄存器FLAGS中

» 总线接口单元



» 总线接口单元

功能：

顺序执行

A

从内存中取指令到指令队列

指令队列是并行流水线工作的基础

B

负责与内存或I/O接口之间的数据传送

转移执行

C

在执行转移程序时，BIU清除指令队列，从指定的新地址取指令，并立即传给执行单元执行。

» 结论



指令队列的存在使EU和BIU两个部分可
并行工作，即：

实现指令的并行执行



目的：

- 提高了CPU的效率；
- 降低了对存储器存取速度的要求

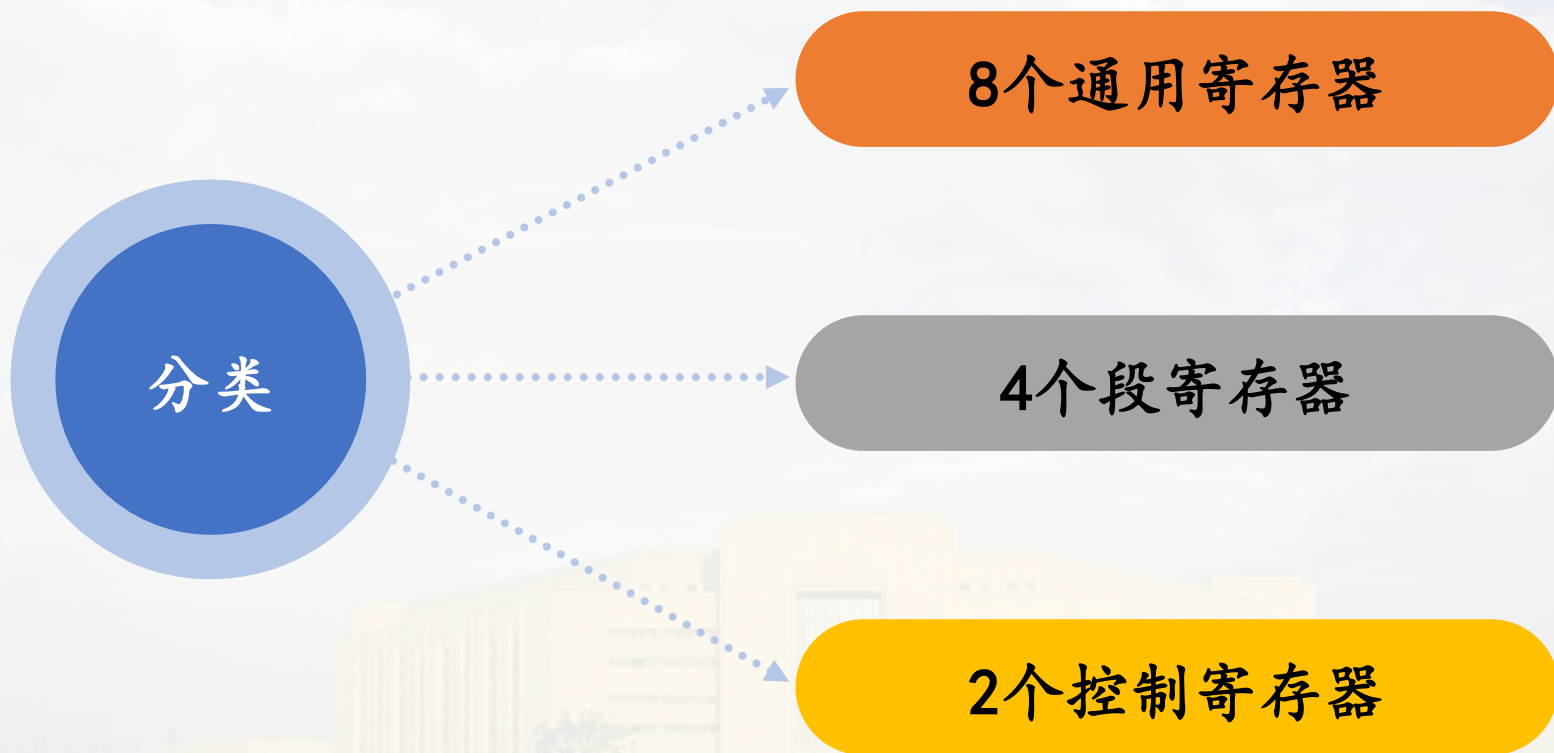
2.4

内部寄存器

2.4 内部寄存器

内部寄存器的类型

含14个16位寄存器，按功能可分为三类



要求：深入理解每个寄存器的作用

通用寄存器



2.4 内部寄存器

» 数据寄存器

8088/8086含4个**16位数据寄存器**，每一个又可拆分为2个8位寄存器，即：

AX  AH, AL 

BX  BH, BL 

CX  CH, CL 

DX  DH, DL 

AX:Accumulator register, BX:Base register, CX:Counter register, DX:Data register

» 数据寄存器特有的固有用法

In AX, n
Out n, AX

AX

累加器。所有I/O指令都通过AX/AL与接口传送信息，中间运算结果也多放于AX中；

BX

基址寄存器。在间接寻址中用于存放基地址；

CX

计数寄存器。用于在循环或串操作指令中存放计数值；

DX

数据寄存器。在间接寻址的I/O指令中存放I/O端口地址；在32位乘法运算时，存放高16位数。

MOV DX, 0FC00H
In AX, DX

2.4 内部寄存器

DX的用法：对外围设备单独编址：编址到寄存器级

端口地址：**8位**，即 2^8 ， $0 \sim 255$ ， $00H \sim FFH$ ，采用**直接寻址**

16位，即 2^{16} ， $256 \sim 64K$ ， $0100H \sim FFFFH$ ，采用**间接寻址**

例1：IN AX, 0CH ; **0CH表示端口号：即地址**

采用直接寻址，输入端口地址0CH的一个字到AX中

例2：MOV DX, 02ECH **DX表示端口地址02ECH**

IN AX, DX ; (IN AL, DX) ? 对否

采用间接寻址，输入端口地址02ECH的一个字到AX中

» 地址指针寄存器



堆栈指针寄存器，其内容为栈顶的偏移地址；



基址指针寄存器，常用于在访问内存时存放内存单元的偏移地址。

对堆栈访问可用**SP**（正常，栈**顶**）

BP（非正常/读，栈**中**间）

SP: Stack pointer register

BP: Base Pointer register

» BX与BP在应用上的区别

相同

- 作为通用寄存器，二者均可用于存放数据；

区别

区别

作为基址寄存器

- 用**BX**表示所寻找的数据在**数据段**
- 用**BP**则表示所寻找的数据在**堆栈段**

变址寄存器



变址寄存器在指令中常用于存放数据在内存中的地址。

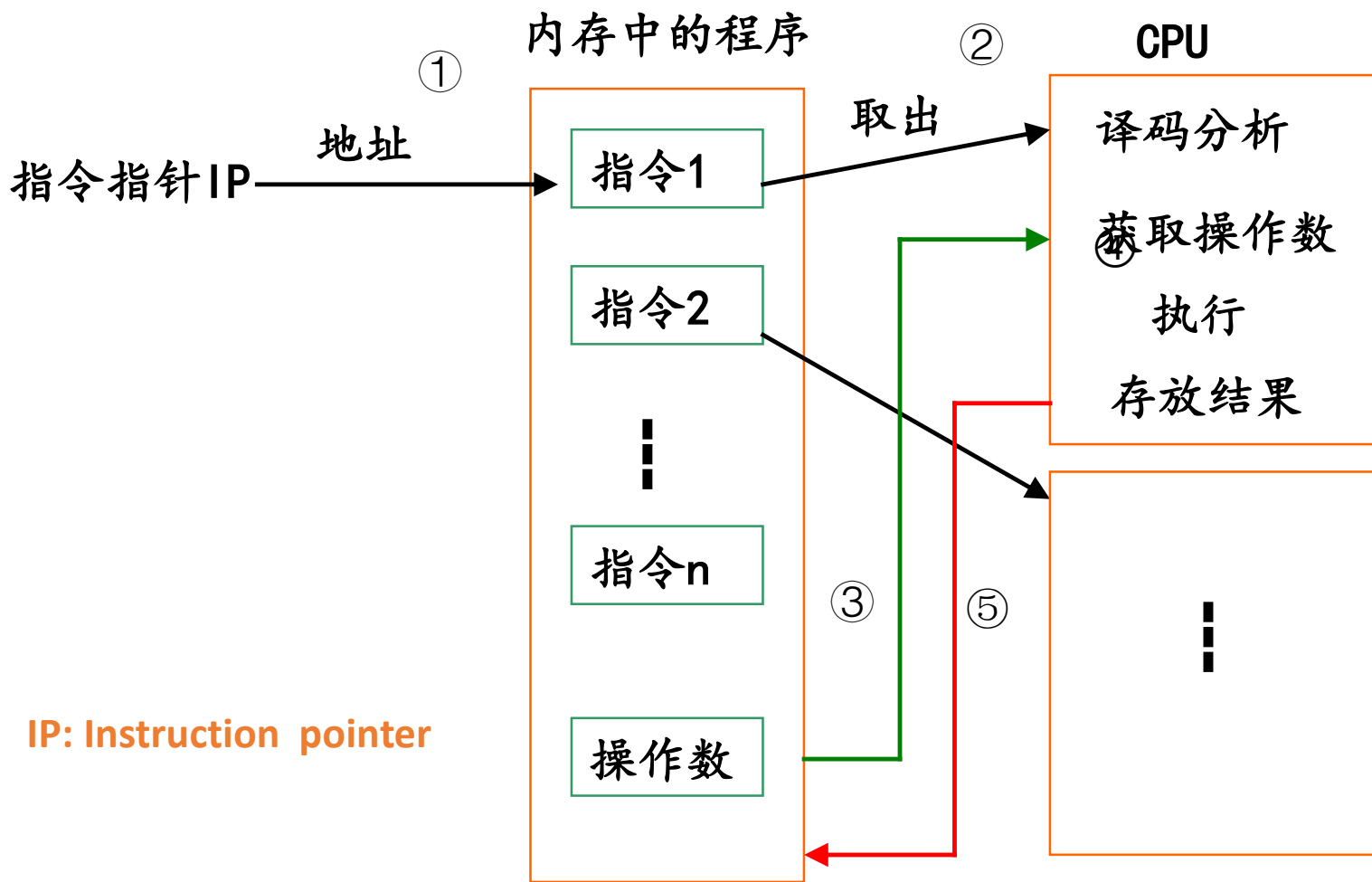
SI: Source index register

DI: Destination index register

2.4 内部寄存器

2个控制寄存器—IP与FLAGS

1) IP指针寄存器:



2.4 内部寄存器

2) FLAGS标志寄存器:



8088 /8086的标志寄存器FLAGS的组成

6个状态位：CF (进位标志位)、PF (奇偶标志位)、AF (辅助进位标志位)、ZF (零标志位)、SF (符号标志位)、OF (溢出标志位)

3个控制标志位：IF (中断允许标志位)、TF (陷阱标志位)、DF (方向标志位)

» 标志寄存器FLAGS ——状态标志位 (1)

若Bit15向Bit16有进位(借位) /
若Bit7向Bit8有进位(借位)

CF (Carry Flag)

进位标志位。加(减)法运算时, 若最高位有进(借)位则CF=1

A

B

奇偶标志位。运算结果的低8位中“1”的个数为偶数时PF=1

PF (Parity Flag)

AF
(Auxiliary Carry Flag)

辅助进位标志位。加(减)操作中, 若Bit3向Bit4有进(借)位, AF=1

C

» 标志寄存器FLAGS ——状态标志位 (2)

ZF (Zero Flag)

零标志位。当运算结果为零时ZF=1

A

B

符号标志位。当运算结果的最高位为1时，SF=1

SF (Sign Flag)

OF (Overflow Flag)

溢出标志位。当算术运算的结果超出了有符号数的可表达范围时，OF=1

C

状态标志位例

给出以下运算结果及运算后各状态标志位的状态：

■ $10110110 + 11110100$

$$\begin{array}{r} 10110110 \\ + 11110100 \\ \hline 1\ 10101010 \end{array}$$

CF = 1 **OF = 0**

AF = 0 **PF = 1**

SF = 1 **ZF = 0**

» 控制标志位



TF (Trap Flag)

陷阱标志位，也叫跟踪标志位。TF=1时，使CPU处于单步执行指令的工作方式。



IF (Interrupt Enable Flag)

中断允许标志位。IF=1使CPU可以响应可屏蔽中断请求。



DF (Direction Flag)

方向标志位。在数据串操作时确定操作的方向。

2.4 内部寄存器

» 4个段寄存器

作用：

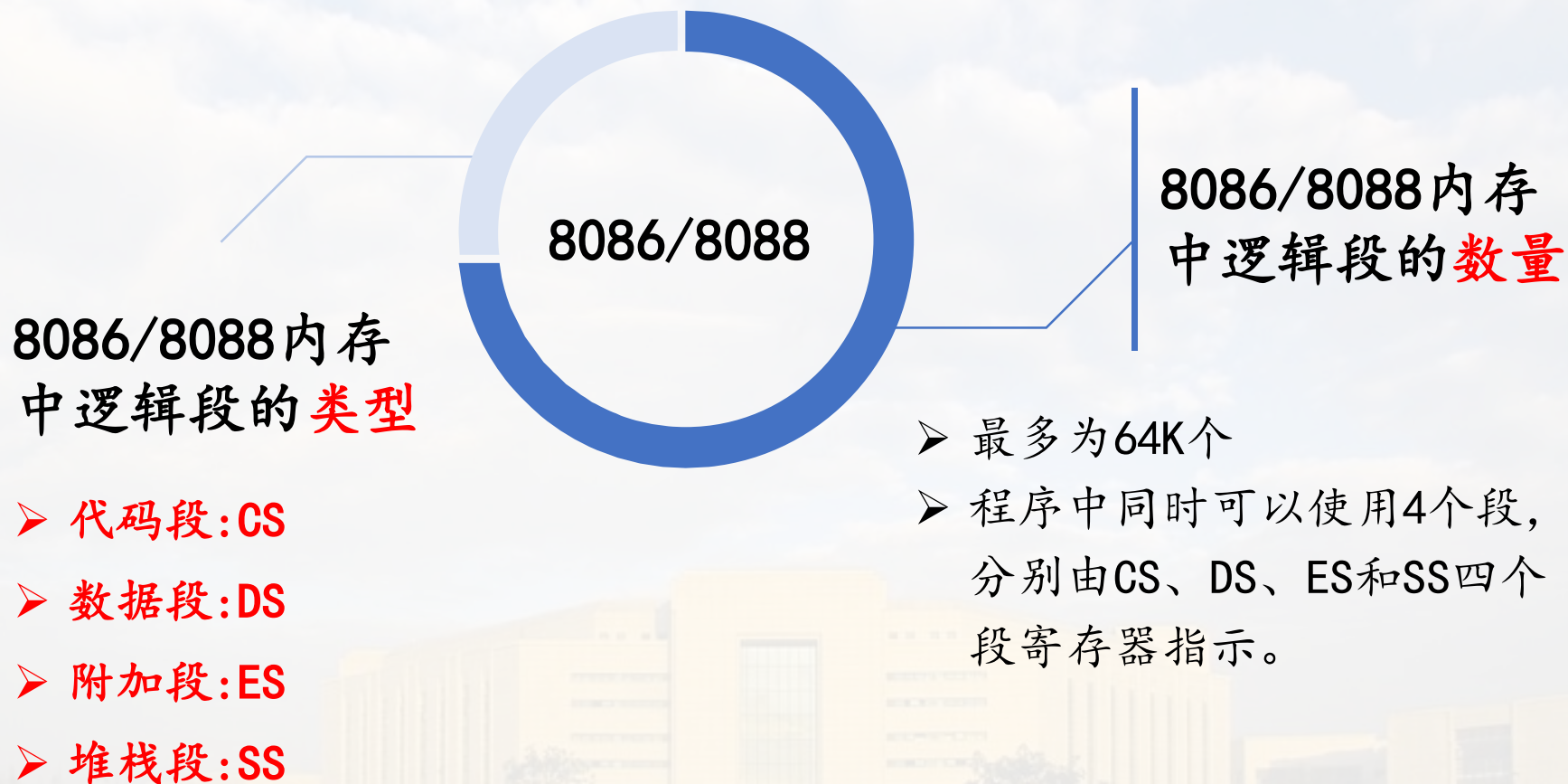
用于存放相应逻辑段的**段基地址**

8086/8088CPU在使用存储器时，将它划分成若干个逻辑段。

每个逻辑段用来存放不同目的内容，如程序代码、数据等等。

每个逻辑段用一个段寄存器来指明该段的起始位置（也叫段基址）。

» 段寄存器



2.4 内部寄存器

» 段寄存器

 **CS**: 代码段寄存器, 存放**代码段**的段基地址。

 **DS**: 数据段寄存器, 存放**数据段**的段基地址。

 **ES**: 附加段寄存器, 存放**数据段**的段基地址。

 **SS**: 堆栈段寄存器, 存放**堆栈段**的段基地址。



2.4 内部寄存器

8086/8088对段存储器使用的约定

序号	内存访问类型	默认段寄存器	可重设的段寄存器	段内偏移地址来源
1	取指令	CS	无	IP
2	堆栈操作	SS	无	SP
3	串操作之源串	DS	ES、SS	SI
4	串操作之目标串	ES	无	DI
5	BP用作基址寻址	SS	ES、SS	按寻址方式计算得有效地址EA
6	一般数据存取	DS	ES、SS	按寻址方式计算得有效地址EA

有效地址EA即《计算机组成原理》课程中的操作数地址

《微机原理》课程中的操作数地址=段基址+段内偏移地址(或EA)

EA: effective address

2.5

8086/8088的 存储器组织

» 内存单元的编址

1. 内存单元的地址表示有物理地址和逻辑地址两种方式

指每个内存单元在整个内存空间中具有的唯一的地址。

物理
地址

8086/8088CPU有20根地址线，它可以产生20位的地址码，寻址范围为 2^{20} ，即1兆字节空间。

这一兆字节存储单元的地址范围为： $\underbrace{00\cdots\cdots 0}_{20\text{位}} \sim \underbrace{11\cdots\cdots 1}_{20\text{位}}$ 。

2.5 8086/8088的存储器组织

内存单元的编址

为了方便书写，在源程序中常用5位十六进制数或一个符号来表示一个存储单元的地址。

书写、编程表示

机器内实际表示

8086/8088

十六进制数地址（5位）

二进制数地址（20位） 存储单元（字节）

00000H

000000000000000000000000B

00001H

000000000000000000000001B

00002H

000000000000000000000010B

⋮

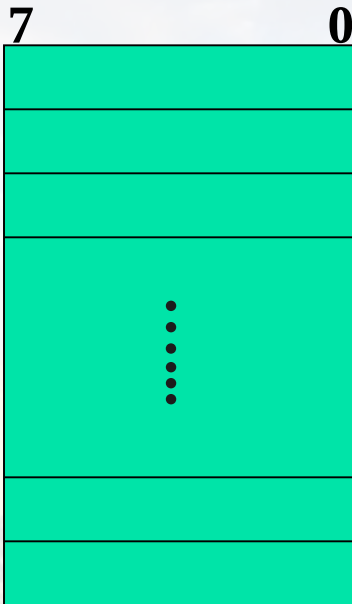
⋮

FFFFEH

111111111111111111111110B

FFFFFFH

111111111111111111111111B



2.5 8086/8088的存储器组织

内存单元的编址



任何两个相邻字节单元就构成一个字单元



字单元的地址为两个字节单元中较小地址字节单元的地址作为字单元的起始地址



字数据的存放规则是低8位放在较低地址字节单元，高8位放在较高地址字节单元

例如，将数据3456H放在地址为09235H的
存储单元中的存储分配。

地址 存储单元

09235H
09236H

⋮
56
34
⋮

字单元

2.5 8086/8088的存储器组织

内存单元的编址

逻辑地址: 每个存储单元的逻辑地址由两部分组成



16位的段（基）地址

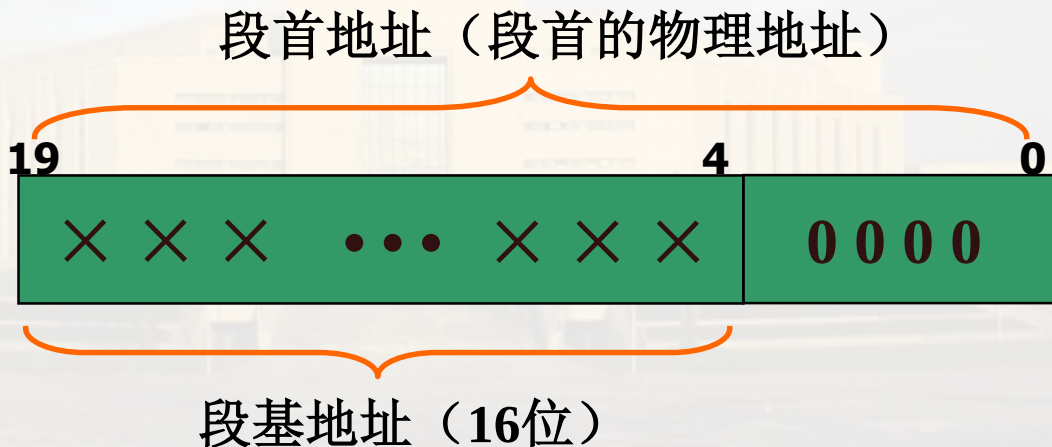
——决定该逻辑段在内存中的位置



16位的段内地址，也叫相对地址，或偏移地址

——决定该存储单元相对段起始单元的距离

逻辑段的起始单元称为段首，段首的偏移地址=0



内存单元的编址



将内存划分成多个逻辑段后，就可以使用逻辑地址来指示存储单元。



使用逻辑地址方便了程序的开发和对存储器进行动态管理。

例：

■ 段基地址 = 6000H

■ 段首地址

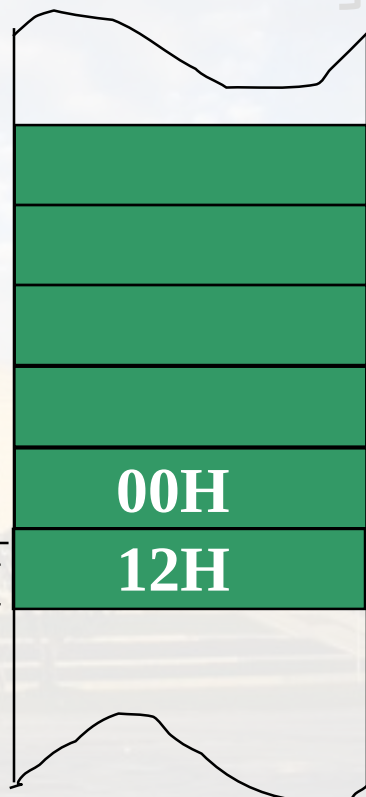
■ 偏移地址 = 0005H

■ 物理地址

6000H

5

6005H



数据段

» 内存单元的编址

01 每个段最大长度为64K
(65536) 个字节单元
组成。

每个段的起始地址
(段首) 必须是一个
小节的首址。 **02**

8086/8088的存储 器段结构的特点

03 逻辑段在物理存储器
中可以是邻接的、间
隔的、部分重叠的和
完全重叠的等4种情况。

在任一时刻，一个程 **04**
序只能访问4类当前段
中某个段的内容。

2.5 8086/8088的存储器组织

» 内存单元的编址

2. 每个段的起始地址（段首）必须是一个小节的首址。

从0地址开始，每16个字节单元称为一个**小节（Paragraph）**
1MB内存就可划分为64K个小节。

第 1 小节: 0000**0**H, 00001H, 00002H.....0000FH

第 2 小节: 0001**0**H, 00011H, 00012H.....0001FH

⋮ ⋮ ⋮ ⋮ ⋮

第65535小节: FFFE**0**H FFFE1H FFFE2H.....FFFEFH

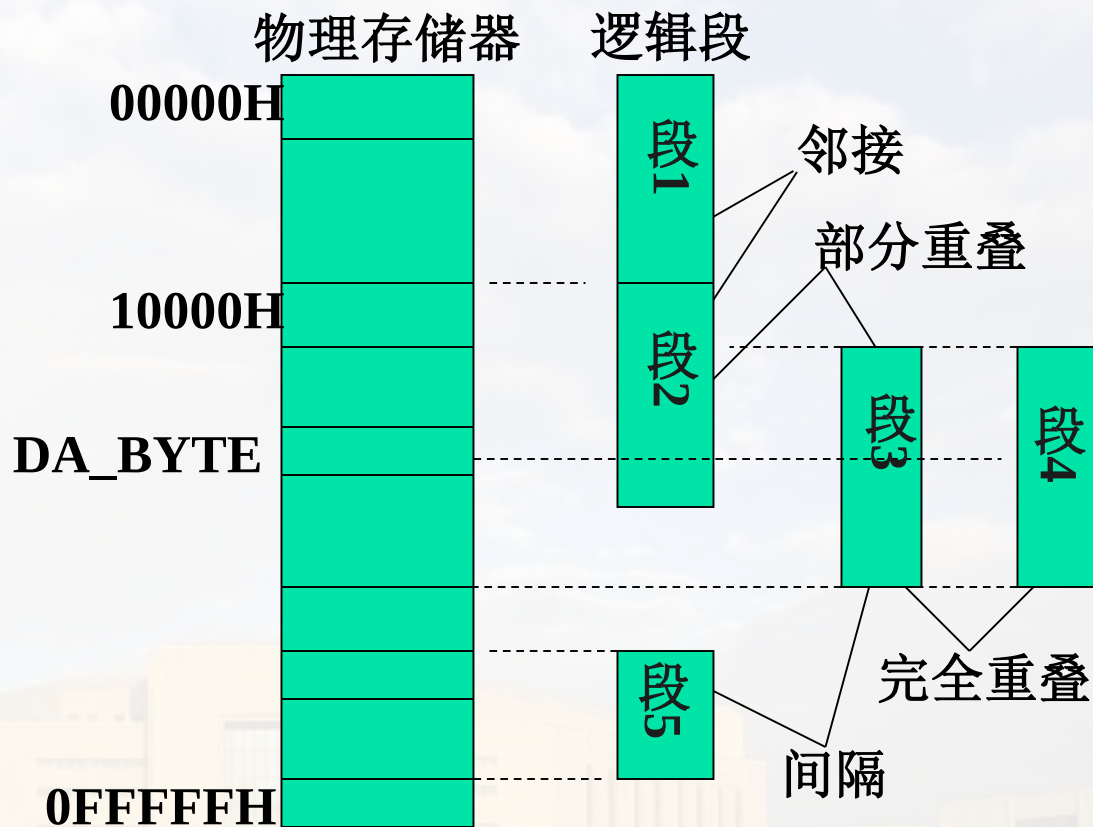
第65536小节: FFFF**0**H FFFF1H FFFF2H.....FFFFFH

每个小节的首地址最低位必为0（16进制数表示）。因此段首只能是上述64K个小节首址之一。

2.5 8086/8088的存储器组织

内存单元的编址

3. 逻辑段在物理存储器中可以是邻接的、间隔的、部分重叠的和完全重叠的等4种情况。



- ◆ 内存中的一个物理存储单元可以映象到多个逻辑段中，可以有多个逻辑地址表示。
- ◆ 例如，DA_BYTE物理单元可以映象到逻辑段2、段3和段4中。

» 内存单元的编址

4. 在任一时刻，一个程序只能访问4个当前段中的内容。

代码段：CS提供当前段的段基地址

数据段：DS提供当前段的段基地址

堆栈段：SS提供当前段的段基地址

附加段：ES提供当前段的段基地址



2.5 8086/8088的存储器组织

内存单元的编址

例：已知 CS=1055H, DS=250AH,

ES=2EF0H, SS=8FF0H

画出各段在内存中的分布。

CS=1055H

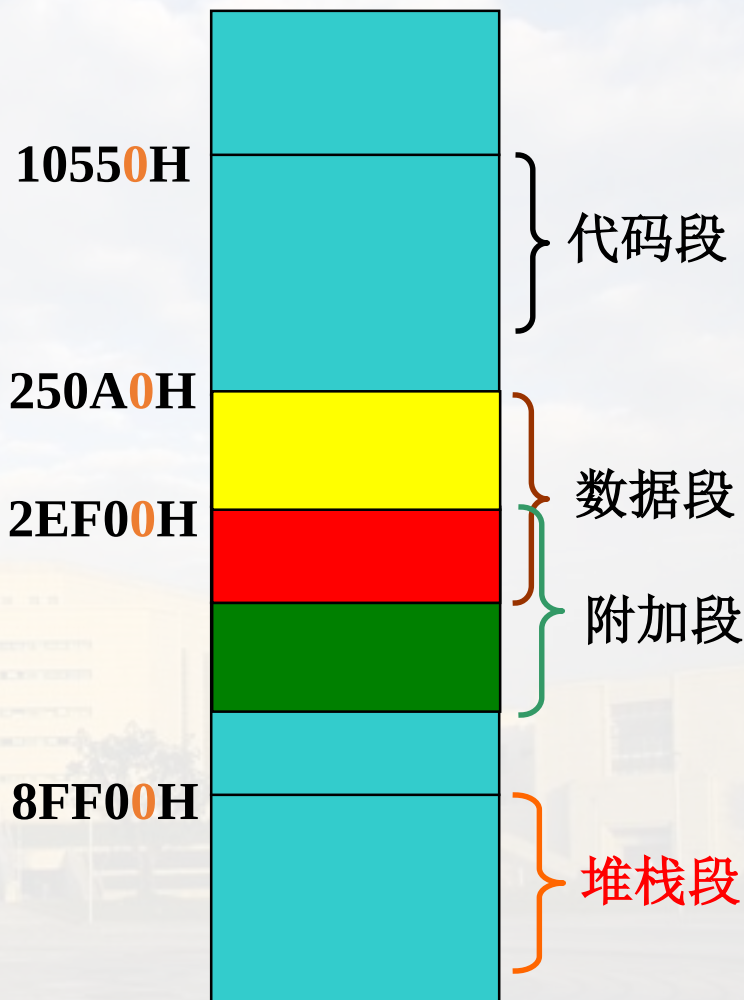
- 段首地址=10550H

DS=250AH

- 段首地址=250A0H

ES=2EF0H?

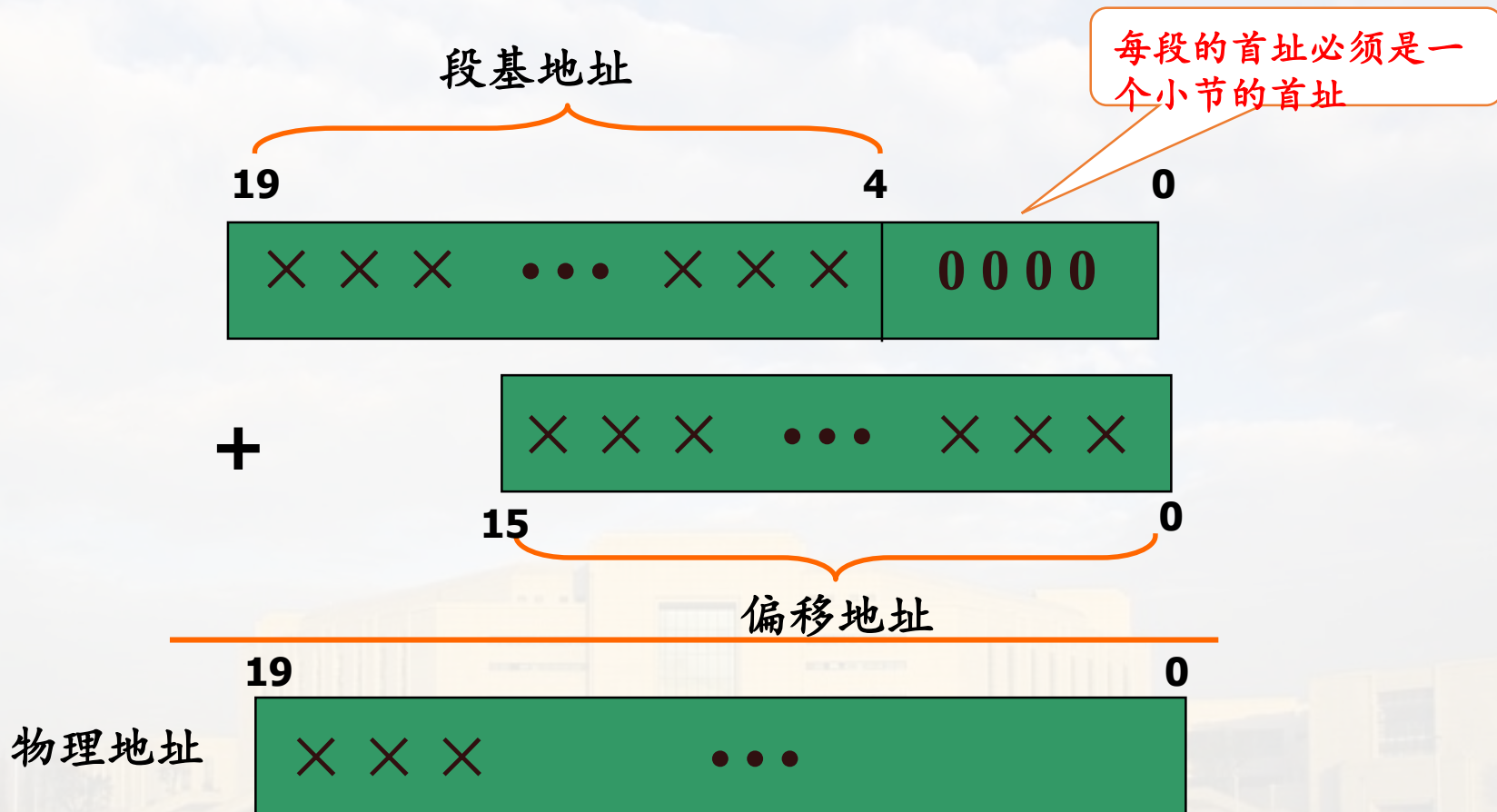
SS=8FF0H?



2.5 8086/8088的存储器组织

逻辑地址与物理地址的转换

内存物理地址由段基址和偏移地址组合而成



物理地址=段基址×16+偏移地址=段基址×2⁴+偏移地址（表示段基址向左整体移动4位）

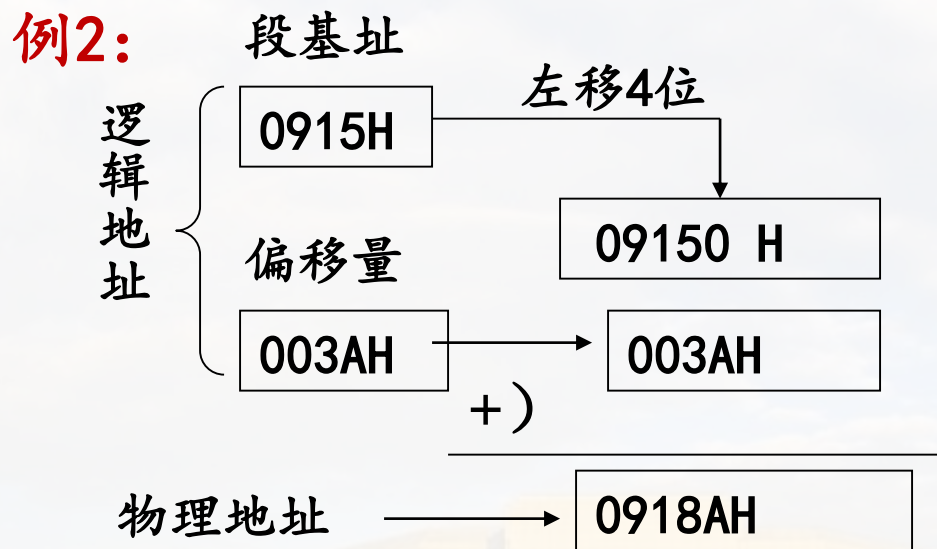
逻辑地址与物理地址的转换

例1

设某操作数存放在数据段，
DS=250AH，数据所在单元的
偏移地址=0204H。则该操作
数所在单元的**物理地址**为：

$$\begin{aligned} & 250\text{AH} \times 16 + 0204\text{H} \\ &= 250\text{A}0\text{H} + 0204\text{H} \\ &= 252\text{A}4\text{H} \end{aligned}$$

逻辑地址与物理地址的转换



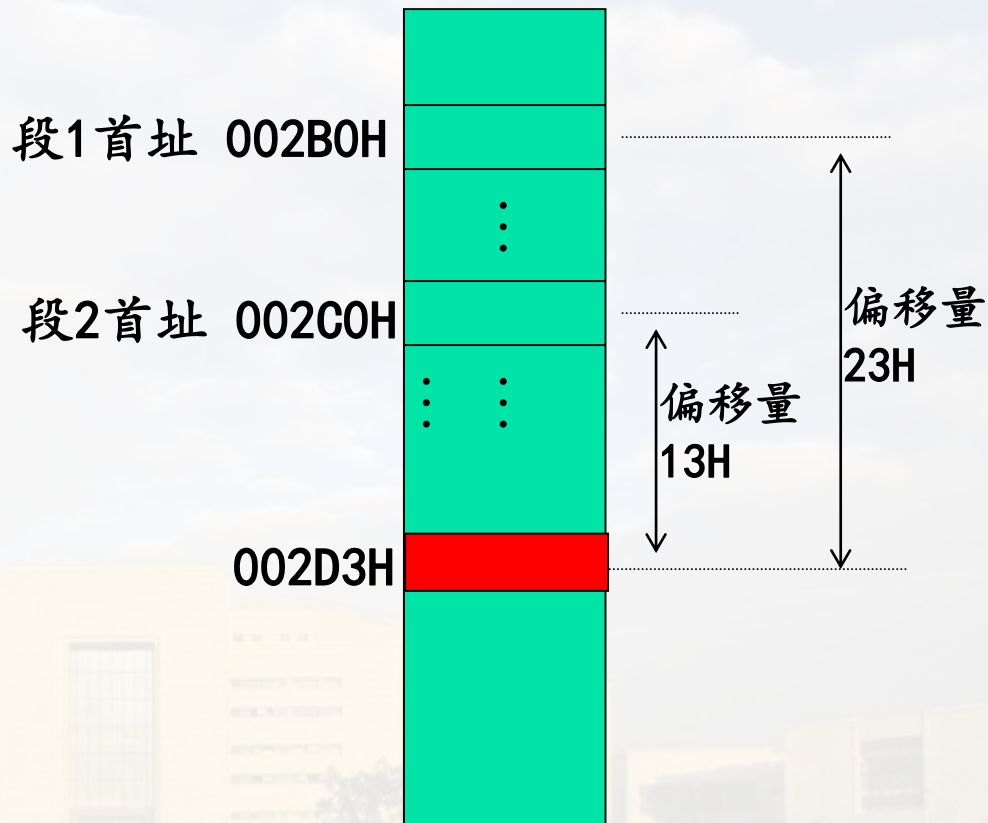
2.5 8086/8088的存储器组织

逻辑地址与物理地址的转换

例3： 同一个物理地址
002D3H被两个逻辑段中的
逻辑地址映射的情况。

$$002B0H + 00023H = 002D3H$$

$$002C0H + 00013H = 002D3H$$



堆栈及堆栈段的使用

堆栈的定义

堆栈是一个特定的存储区，访问该存储区一般需要按照专门的规则进行操作。

主要用于暂存数据以及在过程调用或处理中断时保存断点等信息。

堆栈一般分为**专用堆栈存储器**和**软件堆栈**

按堆栈的工作方式
专门设计的存储器

专用堆
栈存储
器

堆栈

软件
堆栈

由程序设计人员用软件
在内存中划出的一块存
储区作为堆栈来使用。

8086/8088采用这种方式。

2.5 8086/8088的存储器组织

堆栈及堆栈段的使用

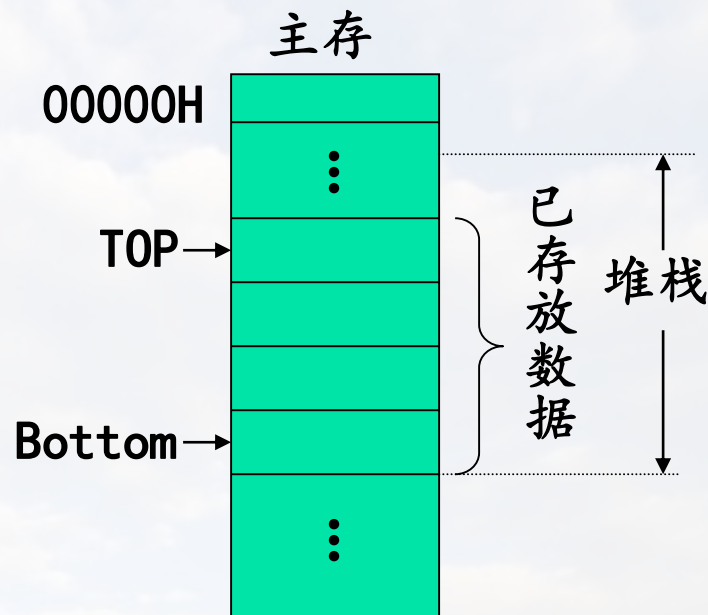
堆栈的一端是固定的，称为**栈底**。

- 栈底是堆栈存储区的**最大地址单元**。



堆栈的另一端是浮动的，称为**栈顶**。

- 任何时候，栈顶是最后存入信息的存储单元。栈顶随着堆栈中存放信息的多少而改变。



堆栈指针SP (Stack Pointer) :

为了指示堆栈中**栈顶**的位置，设置一个寄存器来指示其位置。其内容就像一个指针一样。

SP的内容始终指向**栈顶单元**

堆栈指针
SP

堆栈中数据的**进**
出都由SP来控制

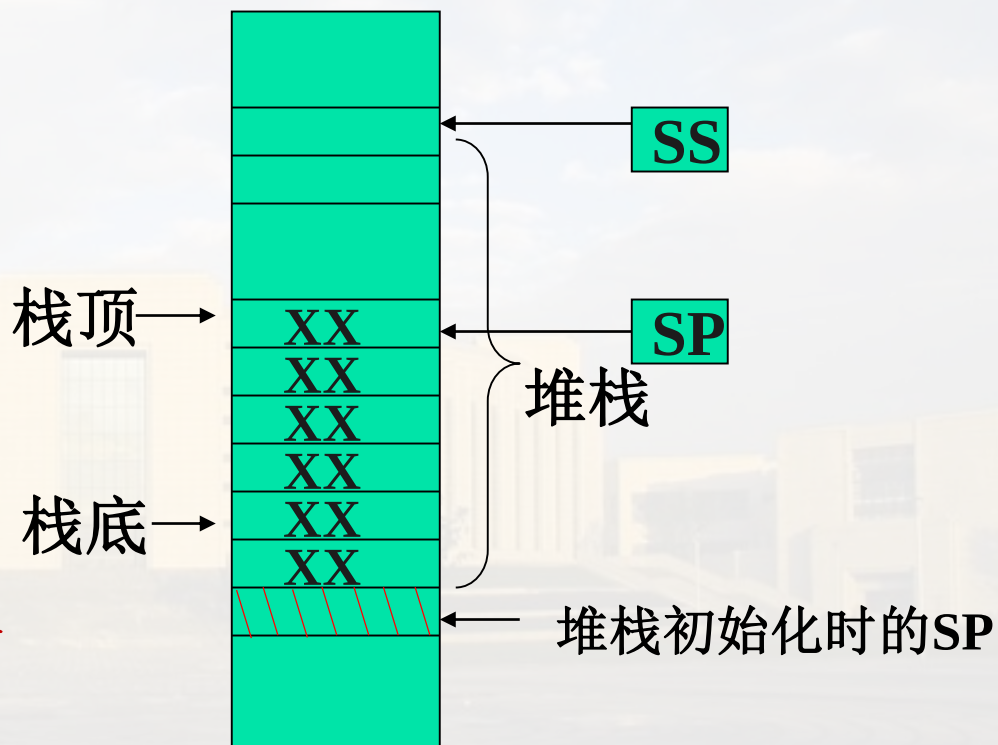
2.5 8086/8088的存储器组织

堆栈及堆栈段的使用

在堆栈中存取数据的规则：“**先进后出FILO**”（First-In Last-Out）。即最先送入堆栈的数据要到最后才能取出，而最后送入堆栈的数据，最先取出。

8086/8088堆栈的组织

在8086/8088微机中堆栈是由堆栈段寄存器SS指示的一段存储区。



注意：栈底是第1个存入数据的地址。

堆栈及堆栈段的使用

01

数据在堆栈中以**字为单位**存放，低8位放在较低地址单元，高8位放在较高地址单元。

02

堆栈初始化时SP指向栈底+2单元，其值就是堆栈的长度。由于SP是16位寄存器，因此堆栈长度 $\leq 64\text{K}$ 字节。

03

SP始终表示堆栈段基址与栈顶之间的距离（字节数）。

- 当SP为最大(初始)值时，表示堆栈为空。
- 当SP为0时，表示堆栈全满。

04

当用户程序中要求的堆栈长度超过一个堆栈段的最大长度64KB时，可以设置多个堆栈段。

2.5 8086/8088的存储器组织

堆栈及堆栈段的使用

例：

■ 已知

■ $SS=1000H$, $SP=0100H$

■ 则：

■ 堆栈段的段首地址= $10000H$

■ 栈顶（偏移）地址= $0100H$

■ 若该段最后一个单元

地址为 $10200H$ ，则：

■ 栈底偏移地址= $0200H$

$10000H$ 段首

栈顶

栈底

堆栈区



2.6

8088 系统总线

2.6 主要内容



- 01. 总线的基本概念
- 02. 最小总线模式
- 03. 最大总线模式
- 04. 最小模式总线时序

概述

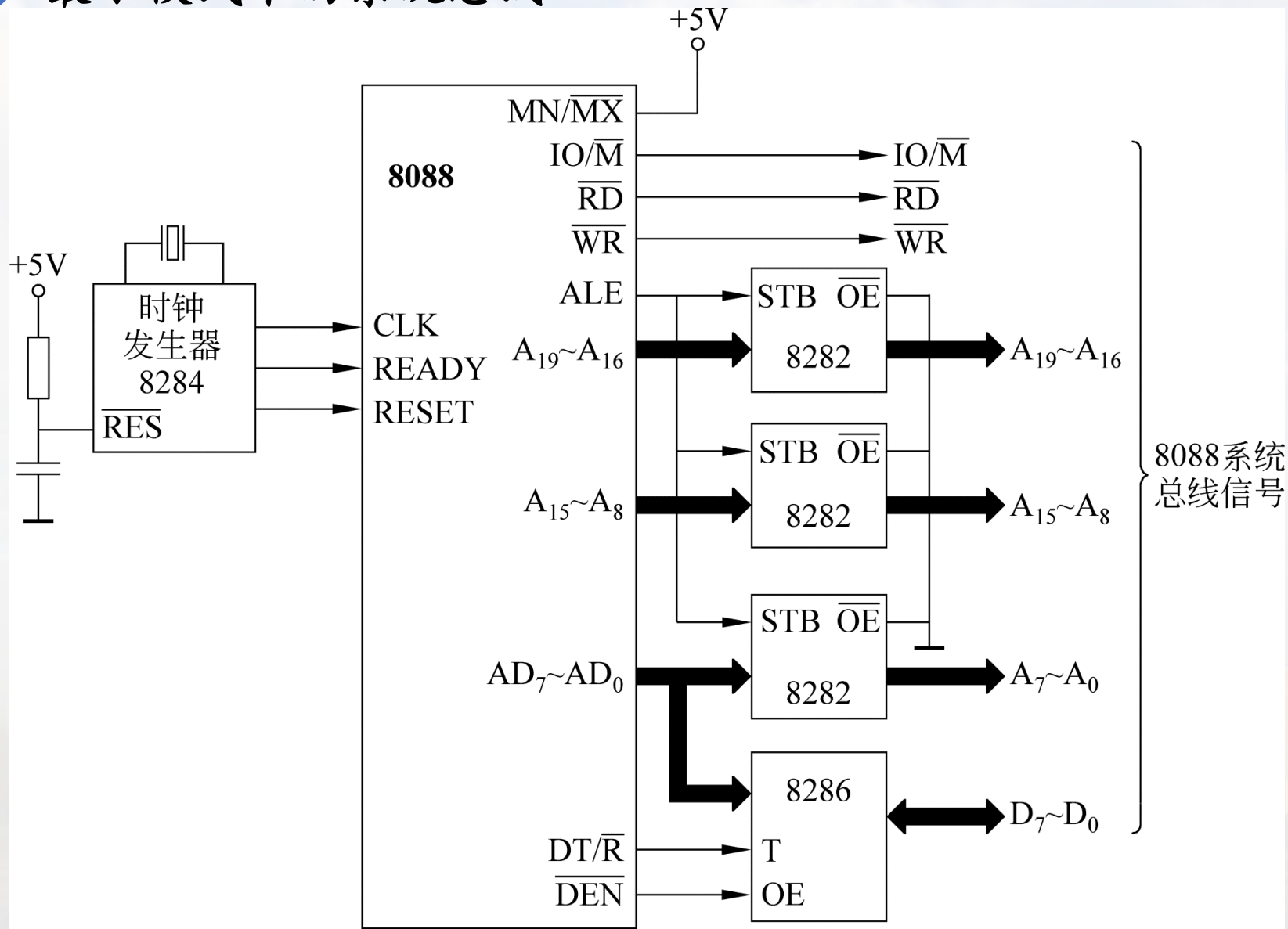
总线：

是一组导线和相关的控制、驱动电路的集合，是计算机系统各部件之间传输地址、数据和控制信息的通道。



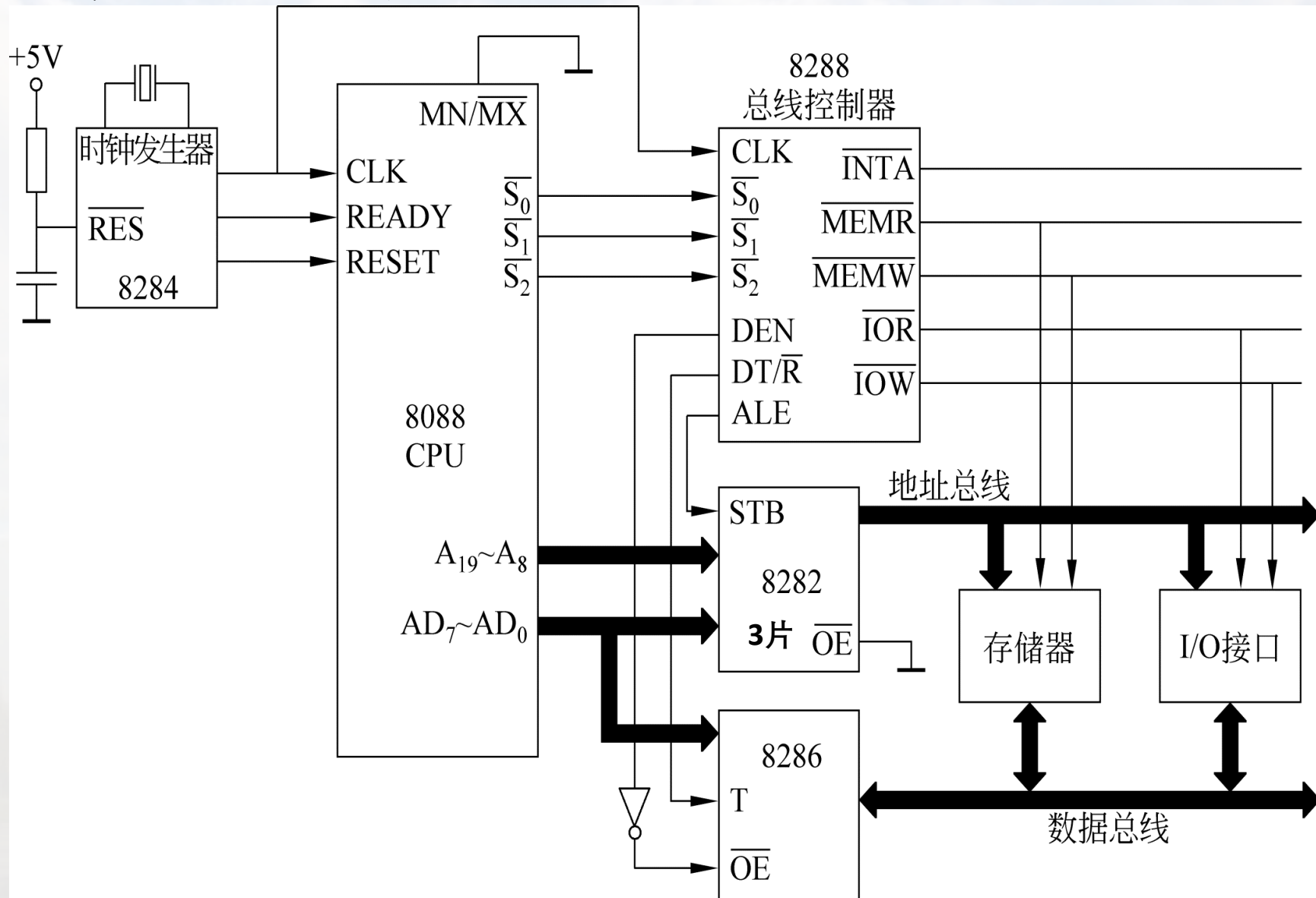
2.6 8088系统总线

最小模式下的系统总线



2.6 8088系统总线

3. 最大模式下的系统总线



» 总线时序

- CPU各引脚信号
在时间上的关系

时序

总线 时序

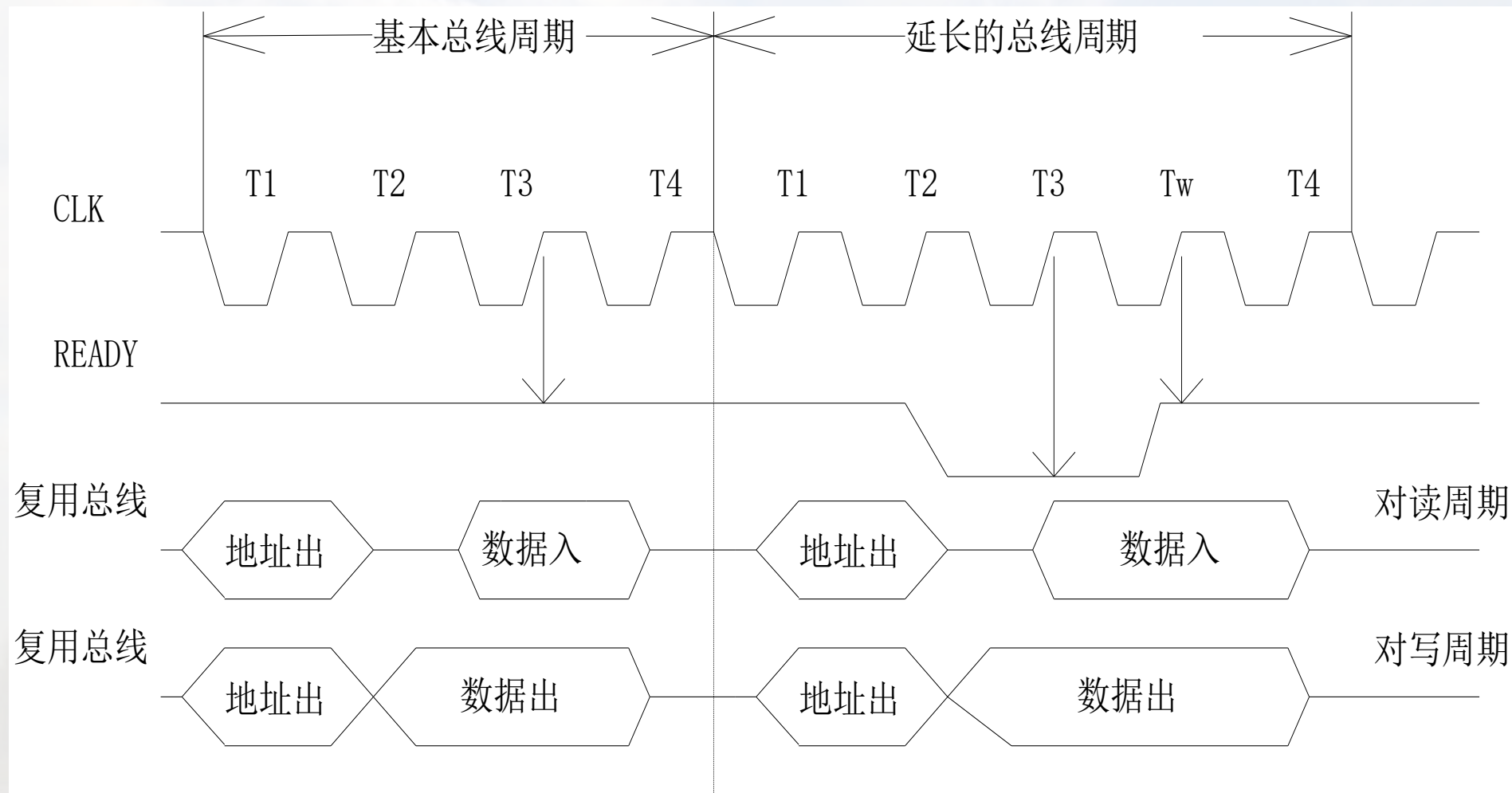
总线周期

- CPU完成一次访问内存（或接口）操作所需要的时间。
- 一个总线周期至少包括4个时钟周期。

2.6 8088系统总线

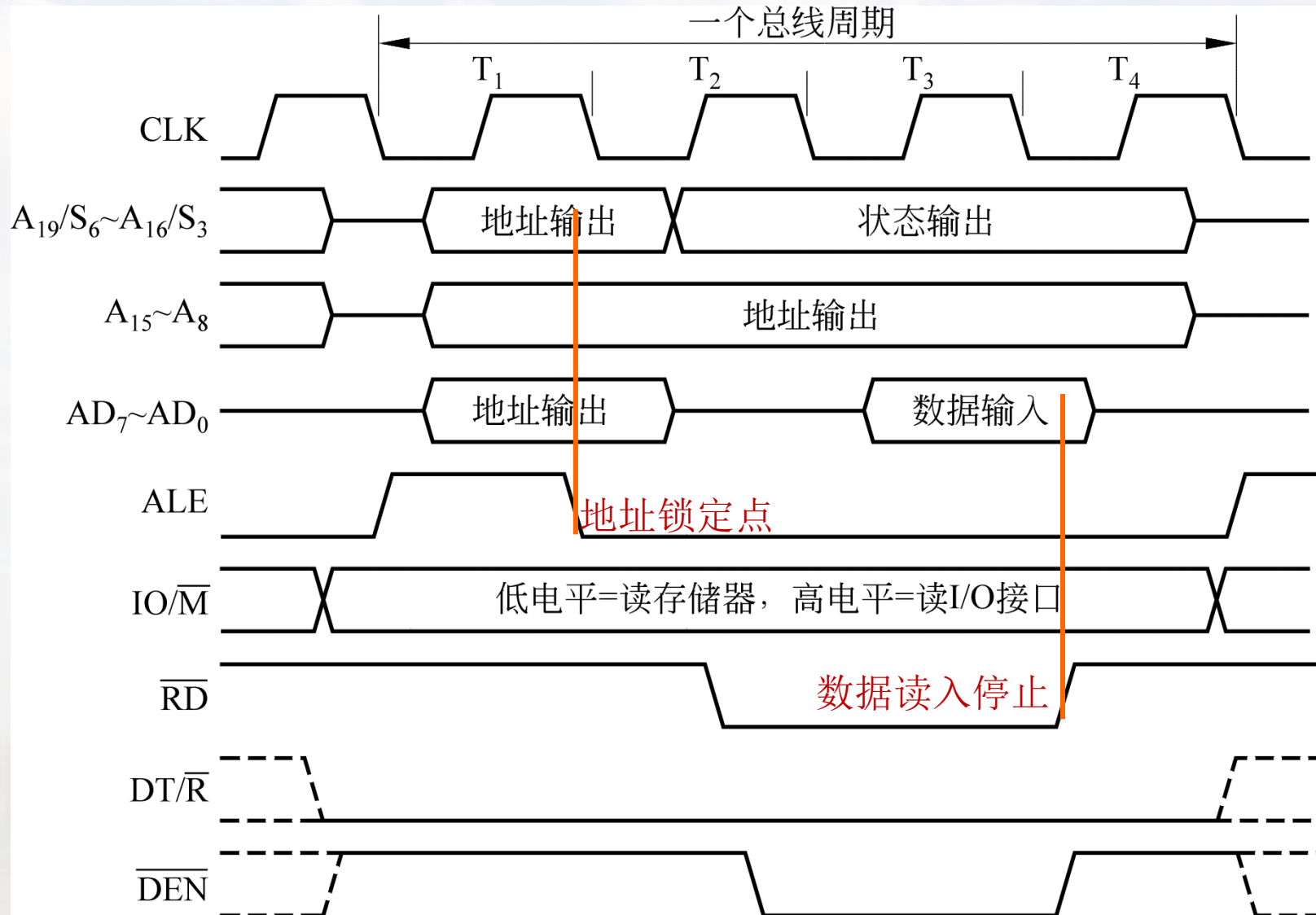
典型的总线周期

进行一次总线操作(访存或访I/O)需4个时钟周期T



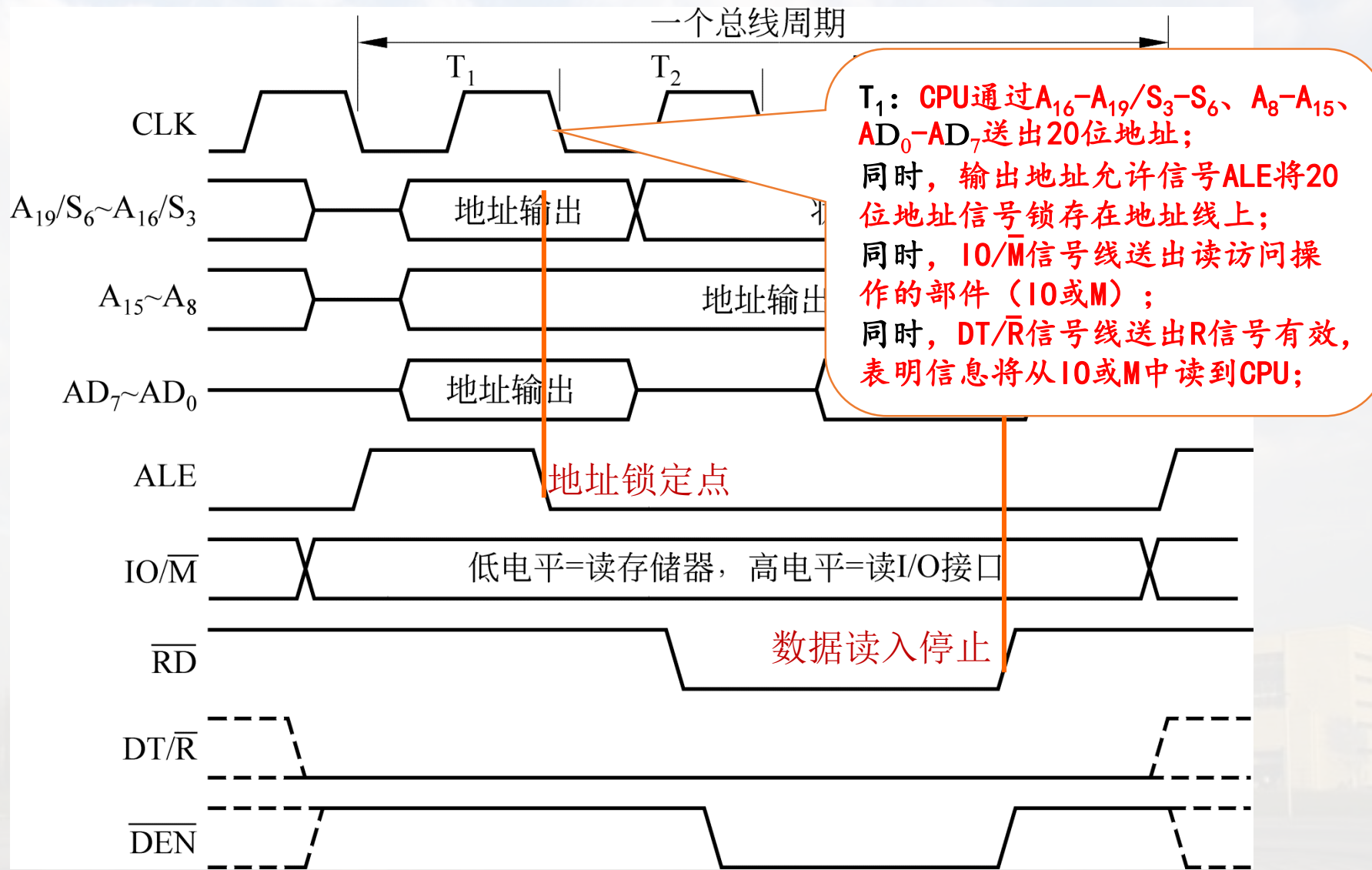
2.6 8088系统总线

8088最小模式下的读周期



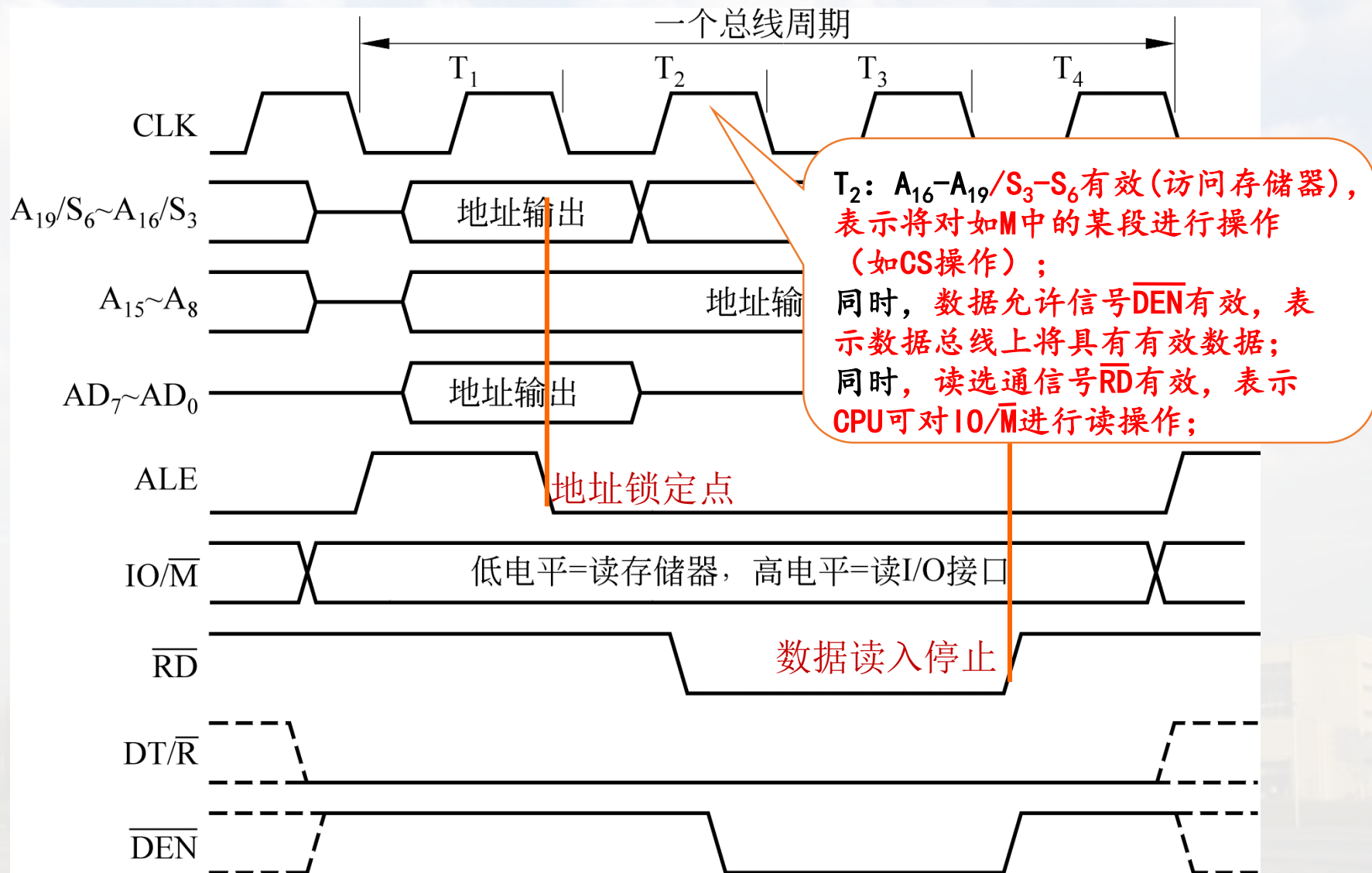
2.6 8088系统总线

8088最小模式下的读周期



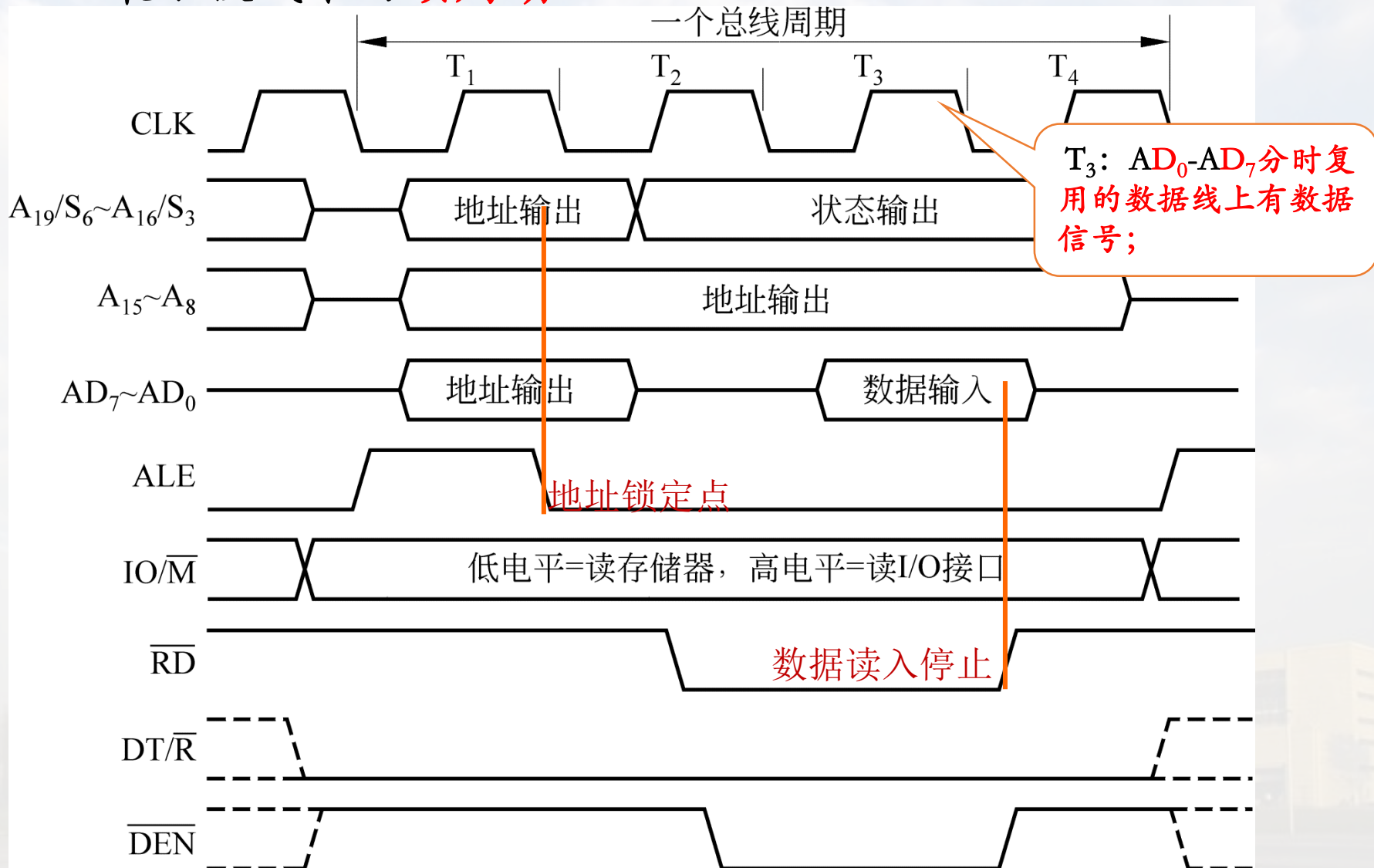
2.6 8088系统总线

8088最小模式下的读周期



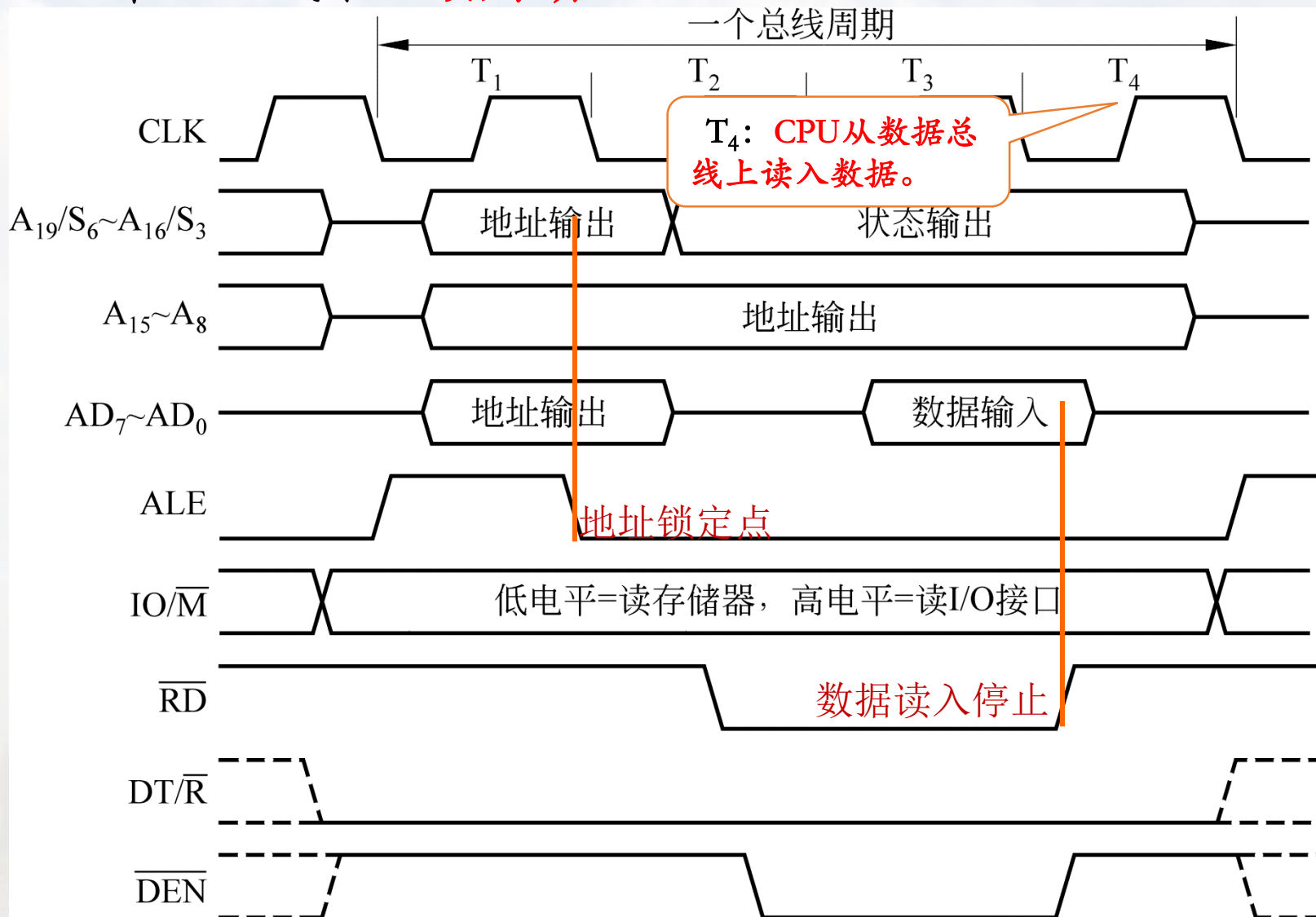
2.6 8088系统总线

8088最小模式下的读周期



2.6 8088系统总线

8088最小模式下的读周期



8088最小模式下的读周期

T_1 : CPU通过 $A_{16}-A_{19}/S_3-S_6$ 、 A_8-A_{15} 、 AD_0-AD_7 送出20位地址;

同时, 输出地址允许信号ALE将20位地址信号锁存在地址线上;

同时, $IO/\overline{M}$ 信号线送出读访问操作的部件 (IO或M) ;

同时, $DT/\overline{R}$ 信号线送出 $\overline{R}$ 信号有效, 表明信息将从IO或M中读到CPU;

T_2 : $A_{16}-A_{19}/S_3-S_6$ 有效 (如果访问存储器), 表示将对如M中的某段进行操作 (如CS操作) ;

同时, 数据允许信号 $\overline{DEN}$ 有效, 表示数据总线上将具有有效数据;

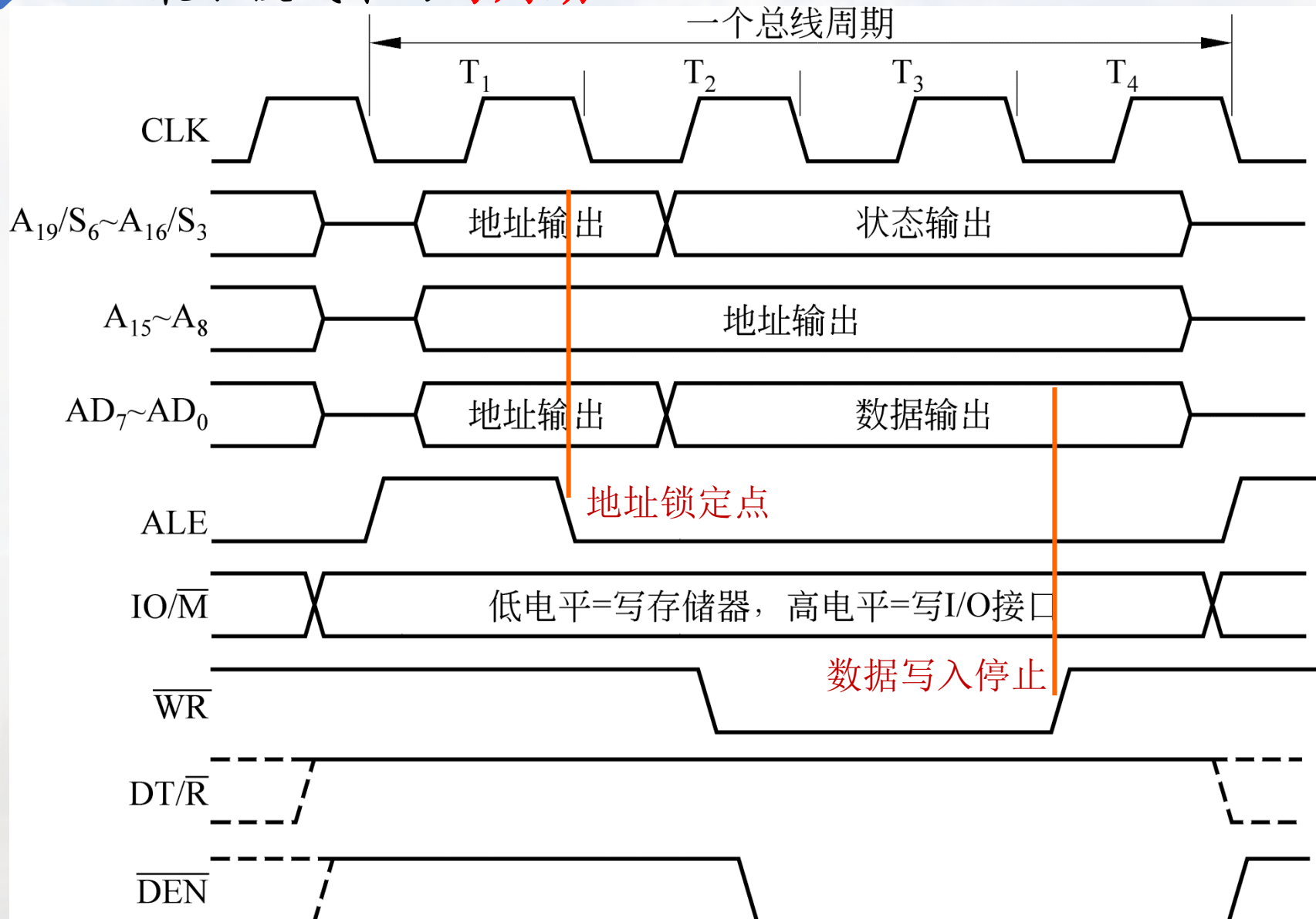
同时, 读选通信号 $\overline{RD}$ 有效, 表示CPU可对 $IO/\overline{M}$ 进行读操作;

T_3 : AD_0-AD_7 分时复用的数据线上有数据信号;

T_4 : CPU从数据总线上读入数据。

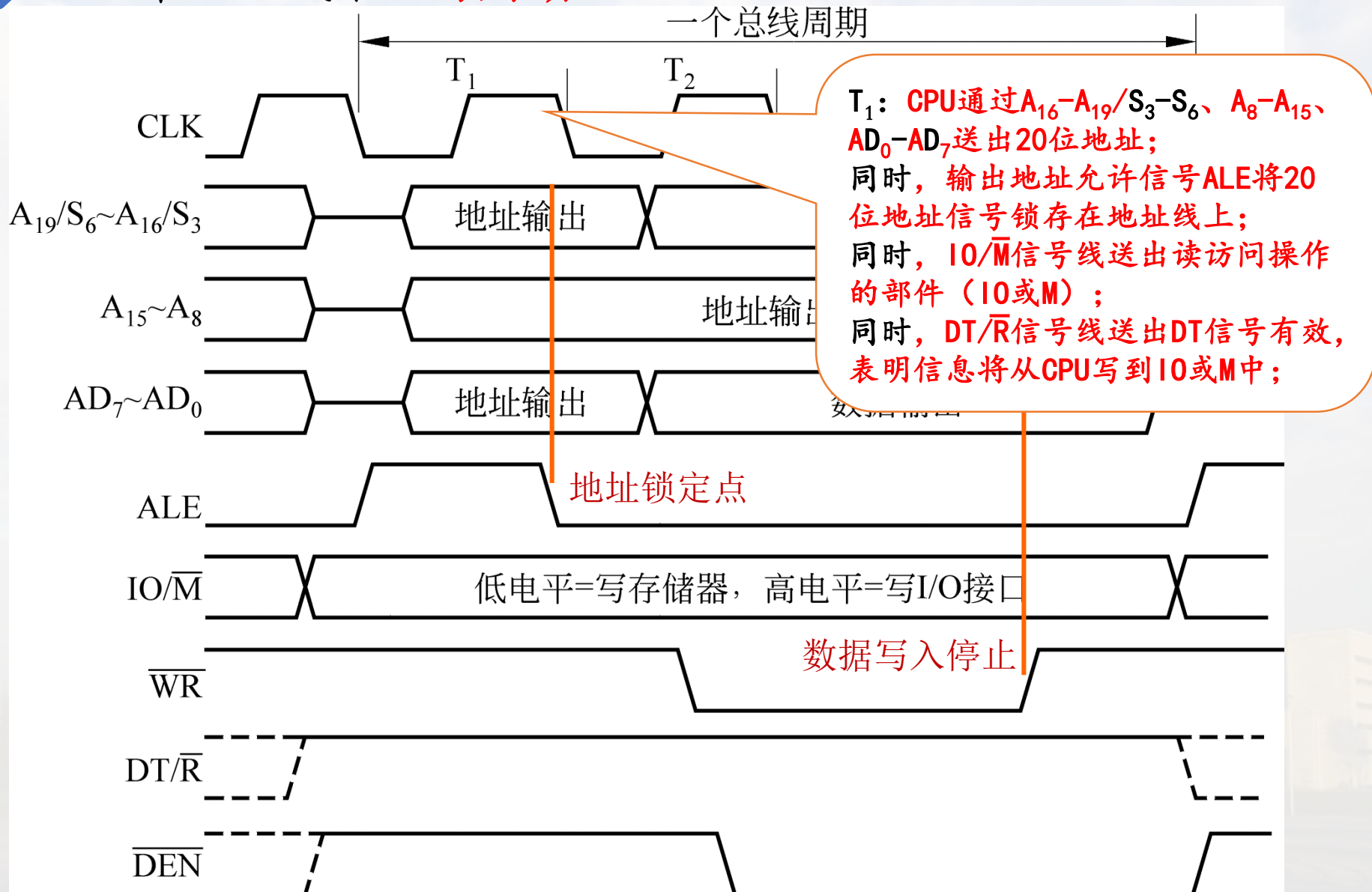
2.6 8088系统总线

8088最小模式下的写周期



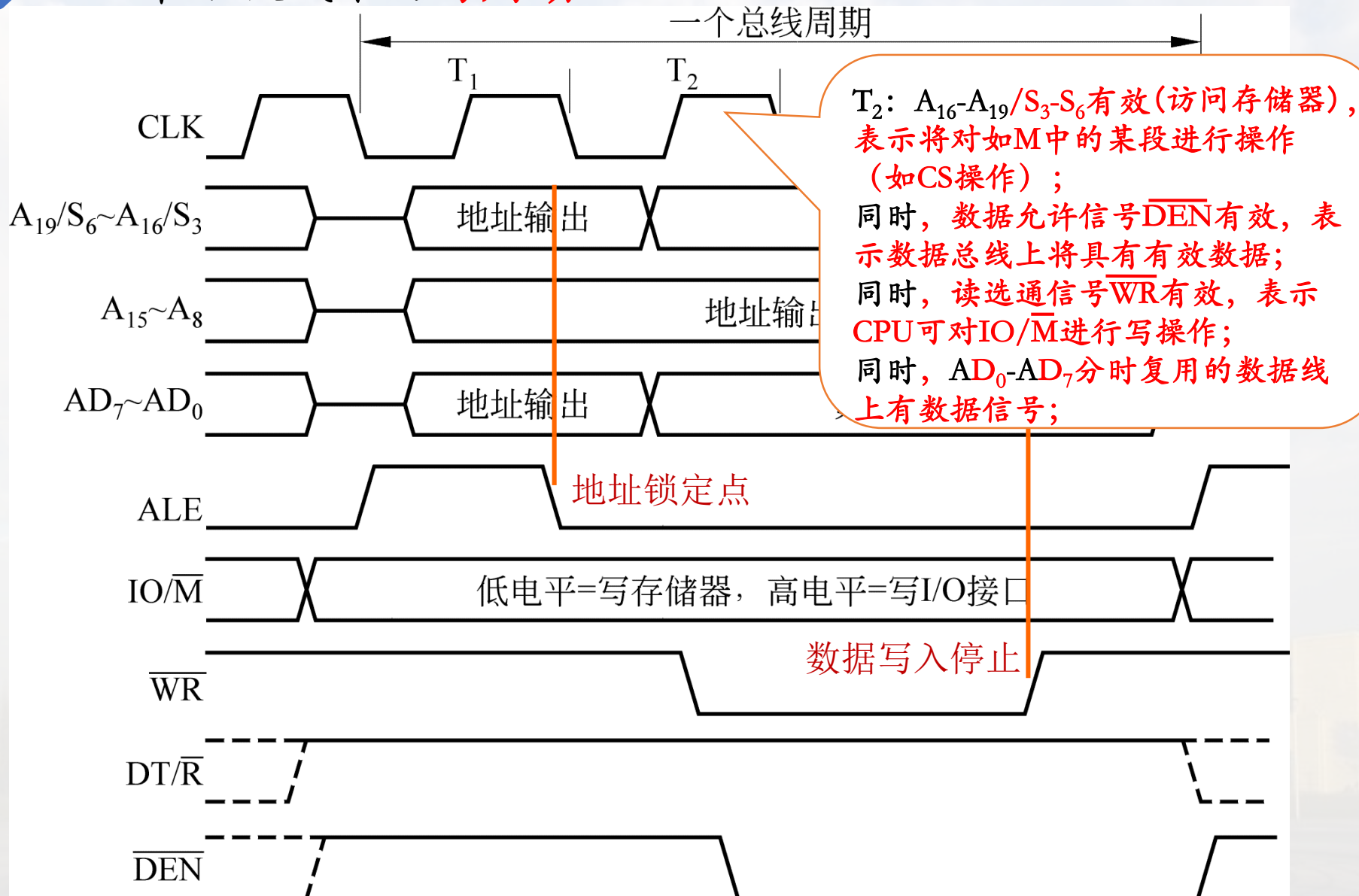
2.6 8088系统总线

8088最小模式下的写周期



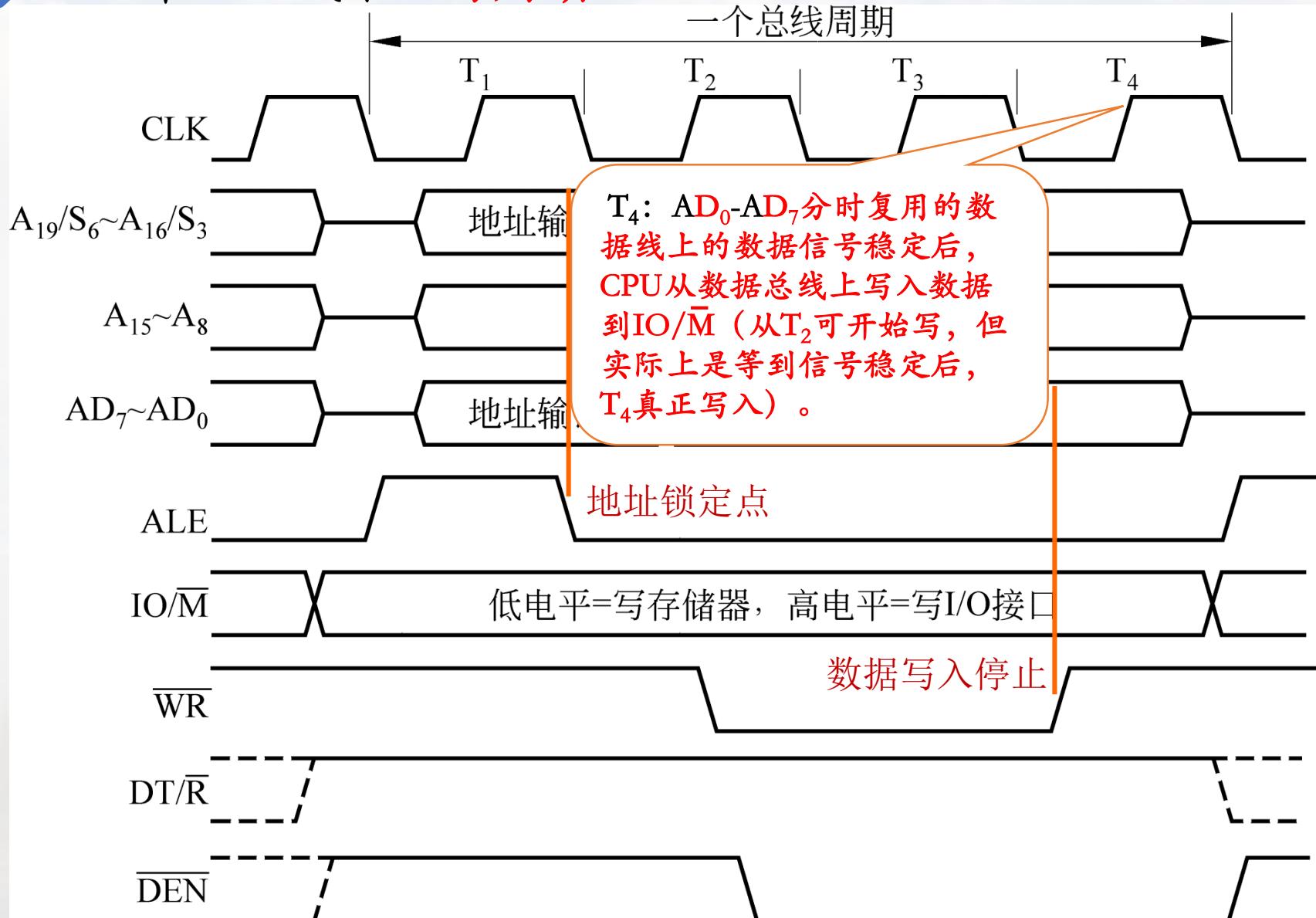
2.6 8088系统总线

8088最小模式下的写周期



2.6 8088系统总线

8088最小模式下的写周期



8088最小模式下的写周期

T_1 : CPU通过 $A_{16}-A_{19}/S_3-S_6$ 、 A_8-A_{15} 、 AD_0-AD_7 送出20位地址;

同时, 输出地址允许信号ALE将20位地址信号锁存在地址线上;

同时, $IO/\overline{M}$ 信号线送出读访问操作的部件 (IO或M);

同时, $DT/\overline{R}$ 信号线送出DT信号有效, 表明信息将从CPU写到IO或M中;

T_2 : $A_{16}-A_{19}/S_3-S_6$ 有效 (如果访问存储器), 表示将对如M中的某段进行操作 (对M操作);

同时, 数据允许信号 $\overline{DEN}$ 有效, 表示数据总线上将具有有效数据;

同时, 读选通信号 $\overline{WR}$ 有效, 表示CPU可对 $IO/\overline{M}$ 进行写操作;

同时, AD_0-AD_7 分时复用的数据线上有数据信号;

T_4 : AD_0-AD_7 分时复用的数据线上的数据信号稳定后, CPU从数据总线上写入数据到 $IO/\overline{M}$ (从 T_2 可开始写, 但实际上是等到信号稳定后, T_4 真正写入)。

- ✓ 微处理器的一般构成
- ✓ 8088CPU的主要引线及其功能
- ✓ 8088CPU的内部结构
- ✓ 内部寄存器功能
 - 寄存器中数据的含义
 - 8位寄存器中存放的均为运算的数据
- ✓ 存储器组织
 - 逻辑地址，段基地址，偏移地址，物理地址
- ✓ 堆栈
 - 栈顶地址，栈底地址，堆栈段基地址
- ✓ 系统总线
 - 最小模式和最大模式的总线，总线时序



第二章补充习题
2.10、2.12、2.14、2.16

THANK

@刘辉
